



DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,
ELETTRONICA E SISTEMISTICA

Corso di Laurea in Ingegneria Informatica

Tesi di Laurea

**Dal Phishing Tradizionale al Bypass MFA:
Studio Implementativo delle Tecniche di
Attacco Moderne**

Relatori:

Prof. Angelo Furfaro
Dott. Matteo Lupinacci

Candidato:

Francesco Pio Ceraudo
Mat. 239814

ANNO ACCADEMICO 2024/2025

Ai miei 22 anni,

*per aver imparato a credere in ciò che faccio
e nel valore dei miei sogni.*

*A questo traguardo che non è un punto d'arrivo,
ma un invito a continuare a crescere
con impegno e passione.*

A mia Madre,

*che con la sua forza e la sua dolcezza
ha saputo accompagnarmi in ogni percorso,
ispirandomi con il suo modo di amare, di credere
e di non arrendersi mai alle interperie della vita.*

A mio Padre,

*esempio di bontà d'animo e determinazione,
che con la sua presenza e i suoi insegnamenti
mi ha trasmesso il valore dell'impegno, della costanza
e dell'umiltà, valori che porto con me
in ogni passo del mio cammino.*

A Francesca,

*per l'amore e il sostegno silenzioso
in ogni mio passo.*

Indice

Indice	2
Introduzione	5
1 Fondamenti di Sicurezza Informatica	6
1.1 Principi fondamentali	6
1.1.1 Resilienza e cultura della sicurezza	8
1.2 Panorama delle minacce moderne	9
1.3 Gestione degli Accessi	10
1.3.1 Autenticazione basata su Password	11
1.3.2 Multi-Factor Authentication	12
1.3.3 Autenticazione basata su fattori biometrici	13
1.3.4 Autenticazione federata e Single Sign-On	14
1.3.5 Passkey e Autenticazione Continua	16
1.4 Intelligenza artificiale nella sicurezza	18
1.4.1 AIxSEC	19
1.4.2 SECofAI	20
1.4.3 Contromisure tecniche: robustezza, privacy, supply chain	22
1.5 Quadro normativo e linee guida operative	23
1.5.1 Regolamentazione dell'AI	24
2 Sicurezza del Web	25
2.1 Web Security	25
2.2 Vulnerabilità delle applicazioni web	29
2.2.1 Client-side Security	30
2.2.2 Server-side Security	34
2.3 Gestione e dirottamento delle sessioni	37

2.3.1	Vulnerabilità nella gestione delle sessioni	37
	Session Fixation	38
	Session Sidejacking	39
	Cross-Site Scripted Hijacking (XSS-based)	39
	Man-in-the-Middle (MITM)	40
	Adversary-in-the-Middle (AiTM)	41
2.3.2	Contromisure e Risposta agli incidenti	41
2.4	Nuovi trend	43
2.4.1	API Security di prima classe	43
2.4.2	Cloud-native e Kubernetes	44
2.4.3	Zero Trust come modello operativo	44
2.4.4	Zero-day e Patching accelerato	45
3	Ingegneria sociale e Phishing	46
3.1	Ingegneria Sociale	46
3.1.1	Tecniche di base	47
3.1.2	L'elemento psicologico	48
3.1.3	Effetti psicologici e reputazionali	48
3.1.4	Resilienza umana contro l'ingegneria sociale	49
3.2	Phishing	49
3.2.1	Origini ed evoluzione	49
3.2.2	Tipologie principali	50
3.2.3	Impatti sulle organizzazioni	50
3.2.4	Contromisure tecniche	50
3.2.5	Rilevamento e risposta	51
3.3	Progettazione e validazione sperimentale	52
3.3.1	JhonCecio: framework per simulazioni di phishing	52
3.3.2	Confronto con Framework simili	53
	The Social-Engineer Toolkit (SET)	53
	BlackEye	54
	GoPhish	55
	King Phisher	55
3.3.3	Architettura e componenti	56

	Scelte progettuali e limiti	57
3.3.4	Reverse proxy per attacchi AiTM	59
	Setup infrastrutturale: VPS, DNS e TLS	59
	Phishlet: definizione e sviluppo	59
	Phishlet Git: esito funzionante ed evidenze	60
	Messaggio fittizio (lure e-mail)	61
	Phishlet Microsoft 365: esito non funzionante e analisi	64
	Confronto e lezioni apprese	64
	Conclusioni	65
	Bibliografia	67
	Ringraziamenti	77

Introduzione

La sicurezza informatica, anche nota come sicurezza IT, è diventata uno degli aspetti più importanti e abilitanti da quando la tecnologia dell'informazione è parte della vita quotidiana delle persone, delle aziende, e degli stati. Senza fiducia nell'identità delle entità coinvolte nelle operazioni informatiche, la sicurezza dei processi collassa o si sposta verso soluzioni meno efficienti. Nel web, questa esigenza si concentra su tre piani intrecciati: comunicazioni sicure, progettazione e sviluppo del codice applicativo orientato alla sicurezza e gestione delle sessioni web. A complicare il quadro intervengono l'evoluzione delle nuove tecnologie come il *cloud-computing* e l'Intelligenza Artificiale Generativa che rappresentano sia soluzioni per rafforzare la difesa, sia soluzioni che introducono nuove superfici di attacco sfruttabili. Inoltre, l'uomo resta sempre il punto più vulnerabile di ogni sistema informatico e l'interesse a sfruttare minacce come il phishing industrializzato e le tecniche di *Adversary-in-the-Middle*) cresce costantemente.

Questa tesi, articolata in tre capitoli, propone dapprima una disamina del dominio teorica della sicurezza informatica dai principi fondamentali all'analisi dell'impatto introdotto dall'intelligenza artificiale. Il secondo capitolo si concentra sulle principali vulnerabilità delle applicazioni web. Infine, il terzo capitolo presenta uno studio implementativo delle tecniche di attacco moderne basate sul phishing con l'obiettivo di dimostrare quali controlli riducono realmente il rischio di *session hijacking* nel web moderno. In particolare, è stato sviluppato **JhonCecio**, un framework leggero per la generazione di *landing* controllate, con architettura a doppio *hop* (tunnel pubblico; proxy stateless; backend). Gli attacchi AiTM sono stati analizzati con *Evilginx* mediante phishlet dedicati.

Capitolo 1

Fondamenti di Sicurezza Informatica

In questo capitolo vengono forniti i fondamenti teorici della sicurezza informatica e il contesto in cui si inseriscono i meccanismi pratici adottati nei sistemi moderni. Dopo una panoramica dei principi (triade CIA e concetti collegati), sono approfonditi i metodi di autenticazione, per poi concludere parlando del ruolo centrale dell'intelligenza artificiale nel ambito della sicurezza.

1.1 Principi fondamentali

La sicurezza informatica non è più materia per soli specialisti: nel 2025 riguarda tutti, dall'utente esperto a chi ha minore alfabetizzazione digitale. Non c'è differenza tra una multinazionale, un piccolo comune che gestisce dati anagrafici o una scuola che usa il registro elettronico: tutti dipendono da sistemi digitali che vanno protetti e monitorati in modo continuo. Anche attività quotidiane come pagare una bolletta online o inviare un'email passano attraverso reti e piattaforme che, se non adeguatamente protette, diventano punti di debolezza. La sicurezza non è quindi soltanto un problema tecnico, ma una questione di **fiducia**. Se i sistemi digitali non ispirano fiducia, i cittadini smettono di utilizzarli, con ricadute dirette sulla vita quotidiana e sull'economia.

Per spiegare le basi della disciplina si fa spesso riferimento alla triade **CIA**: confidenzialità, integrità e disponibilità. Tre parole che riassumono concetti vitali.

- **Confidenzialità:** garantire che solo soggetti autorizzati possano accedere ai dati. Un esempio concreto è la protezione dei registri scolastici online: se un estraneo riesce a visualizzare le informazioni di studenti e famiglie, la confidenzialità è violata.
- **Integrità:** i dati devono rimanere corretti e completi; un attacco che altera anche un singolo valore in una cartella clinica o in una dichiarazione fiscale può avere conseguenze gravi. Nel luglio 2022 il gruppo ransomware *LockBit* ha rivendicato un'intrusione nei sistemi dell'Agenzia delle Entrate, dichiarando l'esfiltrazione di documenti fiscali e aziendali: episodi di questo tipo minano la fiducia nel sistema tributario e sollevano dubbi sull'integrità dei dati [13].
- **Disponibilità:** i servizi devono essere accessibili quando servono. Nel 2023 vari istituti bancari europei hanno subito blocchi temporanei agli sportelli ATM a causa di attacchi DDoS: i dati non erano persi, ma i clienti non potevano accedervi, con disservizi significativi [1, 2].

Accanto alla triade, completano il quadro altri due principi:

- **Autenticità:** la controparte (utente o sistema) deve essere davvero chi dichiara di essere. In assenza di meccanismi di validazione, è facile per un attaccante spacciarsi per un soggetto legittimo (es. phishing). Nei sistemi moderni la garanzia di autenticità è supportata da certificati digitali e firme elettroniche, che fungono da “carta d'identità” crittografica.
- **Non ripudio:** deve essere possibile dimostrare che una certa azione digitale sia stata effettuata proprio da un soggetto specifico. Strumenti quali firma digitale e PEC forniscono evidenze utili in contesti contrattuali, transazioni economiche e processi legali.

La gestione dell'accesso si riassume spesso nel modello **AAA**:

1. **Authentication** — verifica dell'identità tramite credenziali (password, token, certificati, fattori biometrici) e produzione di una sessione o di un token firmato utilizzabile verso servizi a valle.

2. **Authorization** — decisione su quali azioni un soggetto autenticato può eseguire su determinate risorse; nelle architetture moderne è erogata da componenti specializzati (PDP/PEP) e attuata secondo modelli RBAC/ABAC e principi di least privilege e separazione dei doveri.
3. **Accounting** — registrazione e correlazione degli accessi a fini di audit, non ripudio e rilevazione di anomalie; i registri sono tipicamente firmati e integrati con il SIEM.

Il paradigma AAA è applicato in ambienti applicativi, su API e nel piano di rete (es. RADIUS/Diameter su TLS, VPN), fornendo un quadro coerente su cui innestare i controlli descritti nelle sezioni successive. [12]

1.1.1 Resilienza e cultura della sicurezza

In questo contesto, il concetto chiave è diventato la **resilienza informatica**: non basta evitare gli attacchi, occorre saper reagire e riprendersi rapidamente.

1. **Incident response**: piani operativi per gestire gli incidenti in tempo reale;
2. **Backup immutabili**: copie dei dati che non possono essere modificate nemmeno dai ransomware;
3. **Zero Trust Architecture**: un modello che non dà fiducia implicita a nessuno, né utenti né dispositivi, ma richiede controlli continui [21].

Tuttavia, la tecnologia da sola non basta. Uno dei punti più deboli rimane l'**errore umano**. Molti attacchi iniziano con un'email di phishing a cui un dipendente risponde ingenuamente, o con una password debole che non viene cambiata da anni. Anche i mancati aggiornamenti software sono una causa frequente di violazioni.

Per questo, la **formazione degli utenti** è diventata una priorità. In molte università italiane sono stati avviati programmi per sensibilizzare studenti e personale accademico, mentre aziende pubbliche e private organizzano corsi periodici sulla sicurezza.

1.2 Panorama delle minacce moderne

Negli ultimi anni il concetto stesso di minaccia informatica è cambiato parecchio. Non si tratta più, come accadeva in passato, del classico virus che rallenta un computer o che ruba la password di un social: oggi ci troviamo davanti ad attacchi molto più strutturati, pensati per mettere in crisi aziende, enti pubblici e perfino servizi fondamentali per i cittadini. Per rendere l'idea, possiamo guardare a tre tipologie che hanno fatto parlare molto di sé: i ransomware, gli attacchi alla supply chain e quelli diretti contro le infrastrutture critiche.

- **Ransomware:** questi malware cifrano i dati della vittima e li rendono inutilizzabili finché non viene pagato un riscatto, quasi sempre in criptovaluta. Non è solo una questione economica: un attacco simile può bloccare l'attività di intere strutture. L'attacco alla Regione Lazio nel 2021 è stato uno dei casi più noti, ma episodi ancora più recenti hanno coinvolto ospedali in Emilia nel 2024, con visite e interventi rimandati a tempo indeterminato [28, 15].
- **Supply chain attack:** invece di colpire direttamente l'azienda bersaglio, l'attaccante prende di mira un software o un servizio esterno che quell'azienda utilizza. È un metodo subdolo, perché sfrutta la fiducia riposta nei fornitori [16].
- **Infrastrutture critiche:** energia elettrica, trasporti, banche e ospedali non sono più al sicuro come si potrebbe pensare. In questo caso il problema non è solo tecnico o economico: un blocco dei sistemi in questi settori può avere conseguenze immediate sulla sicurezza delle persone. Diversi studi recenti sottolineano come la resilienza delle infrastrutture nazionali sia ormai una priorità [14].

Secondo l'**ENISA Threat Landscape Report 2024**, gli attacchi ransomware in Europa sono aumentati del 200% rispetto al 2022. Un dato che conferma come il problema sia in continua crescita e richieda strategie nuove, non più limitate alla sola prevenzione [14].

1.3 Gestione degli Accessi

Nel contesto dei sistemi informativi, identificazione, autenticazione e autorizzazione sono fasi distinte della gestione dell'accesso.

Identificazione è la semplice dichiarazione di un'identità (per esempio uno username o l'identificativo SPID). Non fornisce prove: è una claim.

Autenticazione è la verifica crittografica della claim tramite una o più credenziali (password, token hardware, challenge su dispositivo, biometria) e produce un livello di assurance dell'identità in linea con linee guida e livelli AAL definiti dal NIST. L'esito genera tipicamente una sessione e/o un token firmato (ad es. SAML/OIDC/JWT) che attesta l'identità verso i servizi a valle. [29]

Autorizzazione è la decisione su quali azioni un soggetto autenticato può eseguire su specifiche risorse (chi può fare cosa su che cosa). Nelle architetture moderne la decisione è delegata a componenti specializzati:

- **PEP** (Policy Enforcement Point): intercetta la richiesta all'applicazione o API e fa rispettare la decisione;
- **PDP** (Policy Decision Point): valuta le policy rispetto a informazioni di contesto (attributi dell'utente, della risorsa, dell'ambiente) e restituisce un esito di tipo permit/deny con eventuali obblighi.

I modelli di autorizzazione più diffusi sono RBAC e ABAC:

- **RBAC** (Role-Based Access Control): i permessi vengono assegnati ai ruoli e i ruoli agli utenti, con un modello composto da utenti, ruoli, permessi, sessioni e, quando serve, gerarchie di ruolo. Le sessioni consentono all'utente di attivare solo i ruoli necessari in un dato contesto operativo, mentre le gerarchie permettono di ereditare permessi (ad esempio un ruolo `admin` include quelli di `editor`), e i vincoli di separation of duties, sia statici sia dinamici, prevengono conflitti d'interesse. Questo impianto favorisce governabilità, audit e campagne di certificazione degli accessi, ma può soffrire di role explosion quando la granularità dei permessi cresce o quando diventano indispensabili condizioni di contesto fini [19].
- **ABAC** (Attribute-Based Access Control): la decisione di accesso è presa dal PDP sulla base di attributi di soggetto, risorsa, azione e ambiente,

valutati con regole booleane e strategie di combinazione come il deny-override(algoritmo di combinazione delle policy,se qualunque regola/policy restituisce Deny, l'esito finale è Deny, anche se altre regole darebbero Permit). In pratica, si esprime la policy come una condizione leggibile, ottenendo controlli granulari, adattivi e consapevoli del contesto, con possibilità di delegation e break-glass controllati; il rovescio della medaglia è una maggiore complessità di definizione, test e audit delle policy e la necessità di una solida attribute governance (provenienza, qualità, aggiornamento degli attributi) [20].

- **Modello ibrido:** molti sistemi combinano i due paradigmi esponendo i ruoli RBAC come attributi (ad esempio `sub.roles`) e demandando la decisione finale ad ABAC per applicare vincoli contestuali su tempo, geografia, livello di rischio o sensibilità della risorsa; in questo modo si conserva l'auditabilità dei ruoli, si evita l'esplosione combinatoria e si ottiene un controllo fine-grained dove effettivamente serve [19, 20].

1.3.1 Autenticazione basata su Password

Il metodo più usato, e allo stesso tempo più discusso, è quello delle **password**. È facile da capire: ognuno sceglie una parola chiave e la usa per accedere. Semplice, immediato, ma con una marea di problemi.

Chiunque abbia un account online sa bene quanto sia complicato ricordarsi tutte le credenziali. La maggior parte delle persone, per comodità, ricicla sempre la stessa password. Questo è un rischio enorme: basta che un sito venga violato e le credenziali finiscono nel dark web. A quel punto gli attaccanti provano la stessa password su altri servizi, ed è così che si verificano i famosi *credential stuffing* [27].

Nel 2022, ad esempio, alcuni servizi comunali italiani hanno subito accessi abusivi perché dipendenti usavano credenziali già compromesse in altri data breach. Nel 2021 come già espresso in parte nella sottosezione precedente la regione Lazio fu bloccata da un ransomware: secondo le prime ricostruzioni, i criminali sfruttarono credenziali non protette adeguatamente [28].

Gli attacchi più diretti restano quelli di *forza bruta*: un programma prova milioni di combinazioni finché non trova quella giusta. Per contrastare questa tecnica, molte piattaforme impongono regole rigide: minimo otto caratteri, simboli, numeri, maiuscole. In teoria aumenta la sicurezza, in pratica spesso genera password difficili da ricordare. Inoltre, password del tipo “P4ssw0rd!” che sembrano complesse, ma sono facilmente indovinabili sono ancora troppo diffuse.

Per ridurre questi rischi, oggi molti browser e sistemi operativi propongono automaticamente password lunghe e robuste, molto più difficili da indovinare rispetto a quelle create manualmente. Soluzioni come il portachiavi iCloud di Apple o i gestori esterni come Bitwarden e 1Password non si limitano a generare password sicure, ma le memorizzano e le compilano in modo trasparente per l’utente. Non tutti si fidano ancora di affidare le proprie credenziali a un software, ma è sempre meno raro che università, enti pubblici e aziende ne raccomandino o impongano l’utilizzo [29].

1.3.2 Multi-Factor Authentication

L’autenticazione a più fattori (MFA) combina categorie diverse di prova — conoscenza (password/PIN), possesso (dispositivo o token), inerenza (biometria) — per alzare il costo dell’attacco oltre ciò che è pratico. Dal punto di vista del rischio, è cruciale distinguere tra metodi *phishable* (trasferibili tramite proxy/AitM) e metodi *phishing-resistant* (vincolati all’origine e al dispositivo).

OTP e canali secondari Gli OTP “one-time password” possono essere basati su contatore (HOTP) o tempo (TOTP). In TOTP, client e server derivano lo stesso codice applicando HMAC su una finestra temporale condivisa e su una chiave segreta pre-distribuita; il codice cambia tipicamente ogni 30 secondi. Questi metodi sono più robusti degli SMS — esposti a rischi di SIM swapping e debolezze del segnalamento — ma restano *phishable* se l’utente inserisce il codice in una pagina di phishing o se è in corso un attacco AitM con rilancio in tempo reale [18, 69].

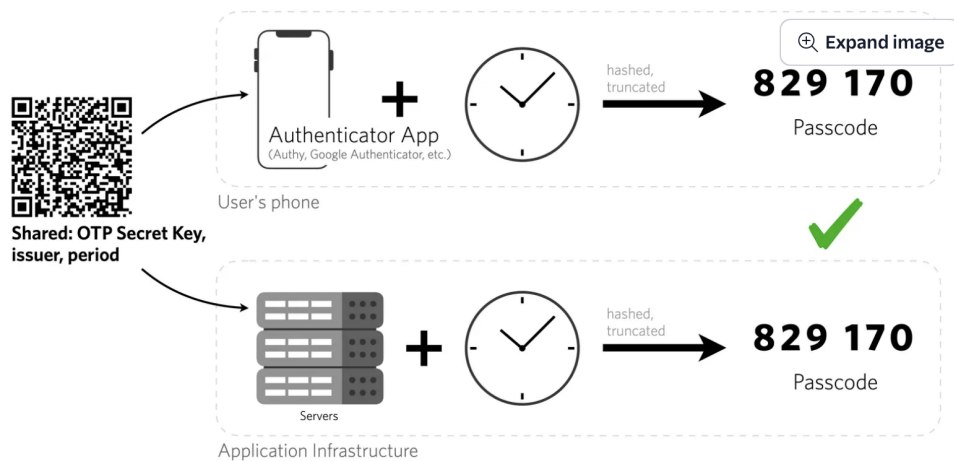


Figura 1.1: derivazione del codice da segreto condiviso e finestra temporale; verifica lato server tramite HMAC. [18].

Push e “number matching” Le notifiche push riducono l’attrito rispetto all’OTP manuale, ma introducono il rischio di *MFA fatigue* (richieste ripetute finché l’utente approva per errore). La variante “number matching” lega la conferma al valore mostrato sul client di origine: l’utente deve digitare nel telefono il numero visualizzato sul dispositivo dove sta facendo login, bloccando approvazioni alla cieca e molti flussi di rilancio [24, 71].

1.3.3 Autenticazione basata su fattori biometrici

Un altro metodo di autenticazione che ormai fa parte della vita quotidiana è la **biometria**, cioè l’uso di caratteristiche fisiche uniche per riconoscere una persona. Non parliamo più di tecnologie futuristiche: basta pensare al gesto di sbloccare lo smartphone con il pollice o con il volto. Anche le app bancarie hanno ormai integrato questi sistemi, così che un bonifico si possa autorizzare con un semplice sguardo alla fotocamera.

La comodità è evidente: niente password da ricordare, tempi di accesso ridotti, maggiore naturalezza per l’utente. Tuttavia, accanto ai vantaggi emergono anche rischi da non sottovalutare. Negli ultimi anni diversi studi hanno mostrato come sia possibile aggirare alcuni sistemi biometrici con tecniche apparentemente banali, come l’uso di maschere stampate in 3D, fotografie ad alta risoluzione o registrazioni vocali [30].

Il problema più delicato riguarda però la **privacy**. Le impronte digitali, i tratti del volto o la voce non sono “password” che si possono sostituire: una volta compromessi, restano esposti per sempre. Questo spiega perché le autorità, come il Garante della Privacy in Italia, abbiano più volte invitato aziende e pubbliche amministrazioni a trattare con estrema cautela i database che raccolgono dati biometrici.

Nonostante queste criticità, la diffusione continua. Gli aeroporti di Fiumicino e Malpensa hanno introdotto procedure di imbarco basate sul riconoscimento facciale: il passeggero non deve più mostrare documenti o biglietti, ma semplicemente “presentare il volto” ai sistemi di controllo. È un esempio chiaro di come la biometria stia entrando in contesti sempre più vari, suscitando allo stesso tempo entusiasmo e preoccupazioni.

1.3.4 Autenticazione federata e Single Sign-On

Come già espresso precedentemente dover gestire troppe password diventa un problema per la nostra memoria, basta pensare a quante volte al giorno ci troviamo a doverci autenticare tra posta elettronica, piattaforme universitarie, cloud e servizi bancari. Ricordarle tutte è quasi impossibile, ed è proprio qui che entra in gioco il **Single Sign-On** (SSO).

Con l'SSO un solo account basta per aprire più porte. Uno studente, ad esempio, può accedere alla piattaforma e-learning, alla mail universitaria e ai servizi del campus senza dover inserire ogni volta nuove credenziali. È come avere un'unica chiave che apre tutte le stanze di un edificio: comodo, veloce, pratico.

Nel Single Sign-On la fiducia tra domini diversi è realizzata tramite tre famiglie di protocolli standard: OAuth 2.0, OpenID Connect e SAML. In tutti i casi il servizio applicativo (RP/SP) non riesegue l'autenticazione: accetta un'asserzione o un token rilasciati da un'autorità di fiducia e ne verifica firma, scadenza e destinatario.

OAuth 2.0 È un framework di delega che consente a un client di ottenere accesso a risorse senza conoscere la password del titolare. Gli attori sono Resource Owner, Client, Authorization Server e Resource Server. Il flusso raccomandato

per applicazioni pubbliche e browser-native è Authorization Code con PKCE: il client invia un code challenge derivato da un verifier, riceve un authorization code sul redirect e scambia il codice al token endpoint presentando il verifier; in questo modo si impedisce il riuso del codice intercettato. I token emessi sono in genere access token a durata breve, limitati da scope, con eventuale refresh token soggetto a rotazione. La redirect URI registrata e il parametro state legano richiesta e risposta e mitigano CSRF e mix-up. Per integrazioni server-to-server si impiega Client Credentials con autenticazione del client tramite secret o mTLS. Il Resource Server valida gli access token verificando firma e claim quando sono JWT oppure mediante introspection, applicando scadenze e politiche di revoca per ridurre l'impatto di compromissioni.

OpenID Connect È un livello di autenticazione costruito su OAuth 2.0. Le parti sono l'OpenID Provider, che autentica l'utente, e il Relying Party, che accetta l'esito. L'OP rilascia un ID Token, firmato come JSON Web Token, che attesta chi è l'utente e per quale RP vale l'asserzione. Il RP verifica la firma con la chiave pubblica esposta dall'OP tramite discovery, controlla emittente, destinatario, scadenza e il valore di nonce legato alla propria sessione. Il flusso operativo più comune è Authorization Code con PKCE: l'utente si autentica presso l'OP, il RP riceve un codice sul redirect e lo scambia in back-channel per ottenere ID Token e, se richiesto, access token. OpenID Connect supporta la registrazione dinamica dei client e identifica l'utente con un soggetto pairwise, diverso per ciascun RP, così da ridurre la correlazione tra servizi.

SAML 2.0 È uno standard di federazione basato su XML. Le parti sono Identity Provider e Service Provider. L'IdP emette una SAML Assertion firmata che il SP valida prima di creare la propria sessione. L'asserzione contiene l'identità dell'utente, le condizioni temporali di validità, il vincolo di audience verso lo specifico SP e una o più affermazioni di autenticazione. Nello scenario più diffuso, avvio dal SP con profilo Web Browser SSO, il SP invia una AuthnRequest all'IdP, l'utente si autentica presso l'IdP e riceve un modulo HTML che pubblica l'asserzione al punto di consumo del SP tramite binding HTTP-POST. Il SP verifica firma, tempi, destinatario e audience e solo allora

abilita l'accesso. Oltre alle asserzioni bearer, SAML può legare la validità alla prova di possesso di una chiave del client con il profilo holder-of-key. [29]

Il vantaggio è evidente: meno password da ricordare, meno rischi di smarrimento o riuso delle stesse credenziali. Ma c'è anche un lato debole: se l'account principale cade in mano a un attaccante, questo ottiene l'accesso a tutti i servizi collegati, non solo a uno. È il classico caso in cui perdere una chiave significa spalancare tutte le porte.

Ecco perché molte aziende, comprese realtà italiane, hanno adottato soluzioni SSO con piattaforme come **Google Workspace** o **Microsoft Azure AD**, ma le affiancano sempre a sistemi di **MFA** garantendo come detto che, anche se la password viene rubata, per entrare serva comunque una seconda verifica[33].

1.3.5 Passkey e Autenticazione Continua

Le passkey realizzano l'accesso senza password usando crittografia a chiave pubblica. Il modello elimina l'inserimento di segreti riutilizzabili e lega l'autenticazione al dominio del servizio, rendendo inefficaci i tentativi di phishing e i proxy in mezzo.

In ambito nazionale, SPID, CIE e CNS coprono l'accertamento dell'identità e gli aspetti legali e organizzativi, mentre le passkey gestiscono l'autenticazione tecnica verso i servizi applicativi. [31]

Registrazione e creazione

1. Il browser invoca l'API WebAuthn per creare una nuova credenziale.
2. L'autenticatore, integrato nel dispositivo o esterno, genera una coppia di chiavi.
3. La chiave pubblica e i metadati di attestazione vengono inviati al servizio (Relying Party) e associati all'account.
4. La chiave privata rimane sul dispositivo, protetta da verifica locale come PIN o biometria.

Questa cerimonia definisce l'associazione tra account, dominio e chiave pubblica, senza mai esporre la chiave privata. [49]

Autenticazione (uso della credenziale)

1. Il servizio invia una sfida crittografica al browser.
2. L'autenticatore firma la sfida solo se il dominio della pagina corrisponde al valore registrato (origin binding).
3. Il servizio verifica la firma con la chiave pubblica memorizzata e completa il login.

Il vincolo di origine impedisce che una credenziale valida per un dominio venga riutilizzata su un dominio diverso, bloccando i tentativi di rilancio tramite attacchi in mezzo. [49]

Componenti FIDO2 FIDO2 unisce due livelli complementari: 1. **WebAuthn**: interfaccia del browser/ambiente applicativo per creare e usare credenziali a chiave pubblica tramite `navigator.credentials.create()` e `navigator.credentials.get()`; 2. **CTAP**: protocollo tra client e autenticatore, che abilita autenticatori integrati (platform) e portatili (roaming) via USB, NFC o BLE.

La separazione tra API web e canale verso l'autenticatore consente di adattare usabilità e sicurezza ai diversi scenari, mantenendo invariato il modello di fiducia e le garanzie crittografiche [49, 50].

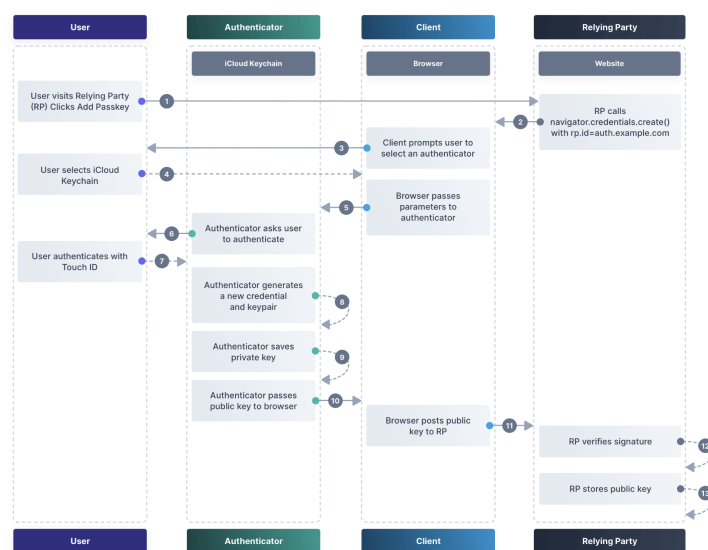


Figura 1.2: Cerimonia di registrazione con WebAuthn e autenticatore.

Origin binding La firma della challenge è consentita solo quando l'origine della pagina corrisponde all'RP ID registrato; una credenziale valida per un dominio non può essere riutilizzata altrove, impedendo il rilancio tramite attacchi in mezzo [49].

Verifica locale L'autenticatore registra la presenza dell'utente e, quando previsto, ne verifica l'identità con PIN o biometria prima di autorizzare l'uso della chiave privata.

Contatore di firma Un contatore monotono aiuta a rilevare riusi anomali e supporta politiche anti-replay; il servizio può individuare regressioni o irregolarità nella sequenza di accessi. L'insieme di questi meccanismi abilita livelli di assurance elevati secondo NIST [29].

Gestione delle credenziali 1. Credenziali residenti e non residenti: le prime sono scopribili dal dispositivo, le seconde sono richiamate tramite identificativo; 2. Passkey multi dispositivo: sincronizzazione sicura tra device dell'utente per migliorare reperibilità e continuità operativa, con politiche di backup e recupero controllate. Queste opzioni permettono di bilanciare praticità e requisiti di sicurezza nei diversi contesti applicativi.[52, 53]

Autenticazione continua e adattiva Oltre al momento del login, la sessione viene rivalutata nel tempo in base al contesto: rete, dispositivo, posizione, segnali comportamentali. Quando il rischio aumenta si applica uno step-up, per esempio la richiesta di una passkey o di un secondo fattore; quando il rischio è stabile si evita di introdurre attrito inutile. Questo approccio riduce gli errori e concentra i controlli dove hanno maggiore impatto. [34]

1.4 Intelligenza artificiale nella sicurezza

L'adozione di sistemi di intelligenza artificiale (AI) sta trasformando la sicurezza informatica lungo due direttrici complementari:

- **AIxSEC (AI for Security)**: uso dell'AI come strumento difensivo per migliorare rilevazione, prioritarizzazione e risposta agli incidenti.

- **SECofAI (Security of AI)**: protezione dell'AI come bene critico, mettendo in sicurezza dati, modelli e pipeline MLOps contro minacce specifiche.

Le due prospettive devono convivere: un motore AI efficace ma non protetto amplia la superficie d'attacco; controlli rigidi senza efficacia operativa degradano la rilevazione.

1.4.1 AIXSEC

Definizione e obiettivi. AIXSEC applica modelli di apprendimento per assegnare punteggi di rischio a eventi e entità (account, host, sessioni, transazioni) e per correlare segnali distribuiti. L'obiettivo è ridurre MTTD/MTTR e rumore per l'analista, con soglie guidate dal costo degli errori.

Flusso tecnico tipico.

1. **Ingest e normalizzazione**: raccolta di log e telemetrie (SIEM, EDR, NetFlow, DNS, proxy), parsing e arricchimento (geo/IP, identità, asset).
2. **Feature engineering e store**: estrazione di contatori, sequenze e grafi di relazione; versionamento delle feature per audit e riproducibilità.
3. **Inferenza e scoring**: modelli non supervisionati (Isolation Forest, autoencoder) rilevano deviazioni; modelli sequenziali (RNN/LSTM) apprendono pattern temporali nei log; classificatori supervisionati coprono casi con etichette storiche.
4. **Decisione e automazione**: soglie ottimizzate rispetto ai costi (C_{FP} , C_{FN}), regole di business, playbook SOAR per contenimento/arricchimento.
5. **Feedback e retraining**: etichette dal SOC (true/false positive) alimentano cicli di miglioramento controllati (canary e rollback).

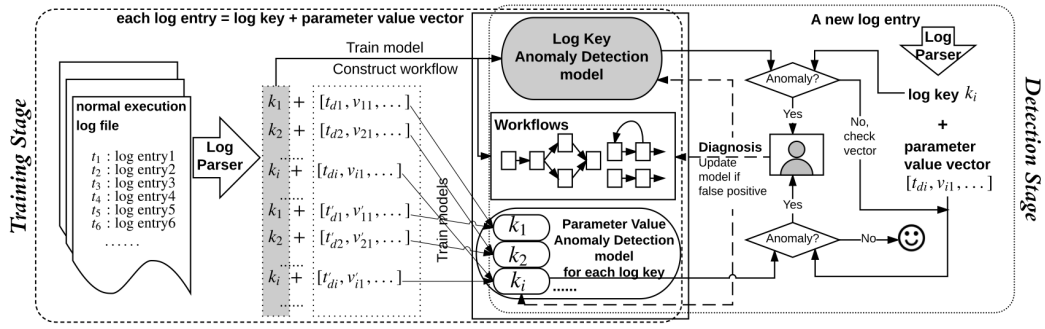


Figura 1.3: Architettura AIxSEC per rilevazione da log (DeepLog). Fonte: [55].

Esempi d'uso concreti. 1. **Rilevazione anomalie e intrusioni:** modellazione delle sequenze di log per individuare deviazioni dalla normale attività di sistema e utente; DeepLog è un riferimento sull'approccio sequenziale alla detection. [55] 2. **Malware e frodi:** classificazione su feature statiche e dinamiche per triage ad alto volume, retro-hunting su archivi e profilazione comportamentale di account e dispositivi. [17]

Valutazione operativa. Con basi altamente sbilanciate si privilegiano precision e recall (curva PR) più della sola AUC; le soglie si fissano minimizzando il rischio atteso $R = \pi_1 C_{FN}(1 - \text{Recall}) + \pi_0 C_{FP}(1 - \text{Precision})$, dove π_1, π_0 sono i tassi base. Le metriche devono riflettere carico analista e impatto sul business. [17]

1.4.2 SECofAI

Definizione e perimetro. SECofAI copre l'intero ciclo di vita dell'AI: dati (raccolta, etichettatura), codice e dipendenze, ambienti di training, artefatti modello, endpoint di inferenza, telemetrie e logging. Il perimetro include confini di fiducia interni/esterni e dipendenze terze (modelli preaddestrati, dataset pubblici, pacchetti).

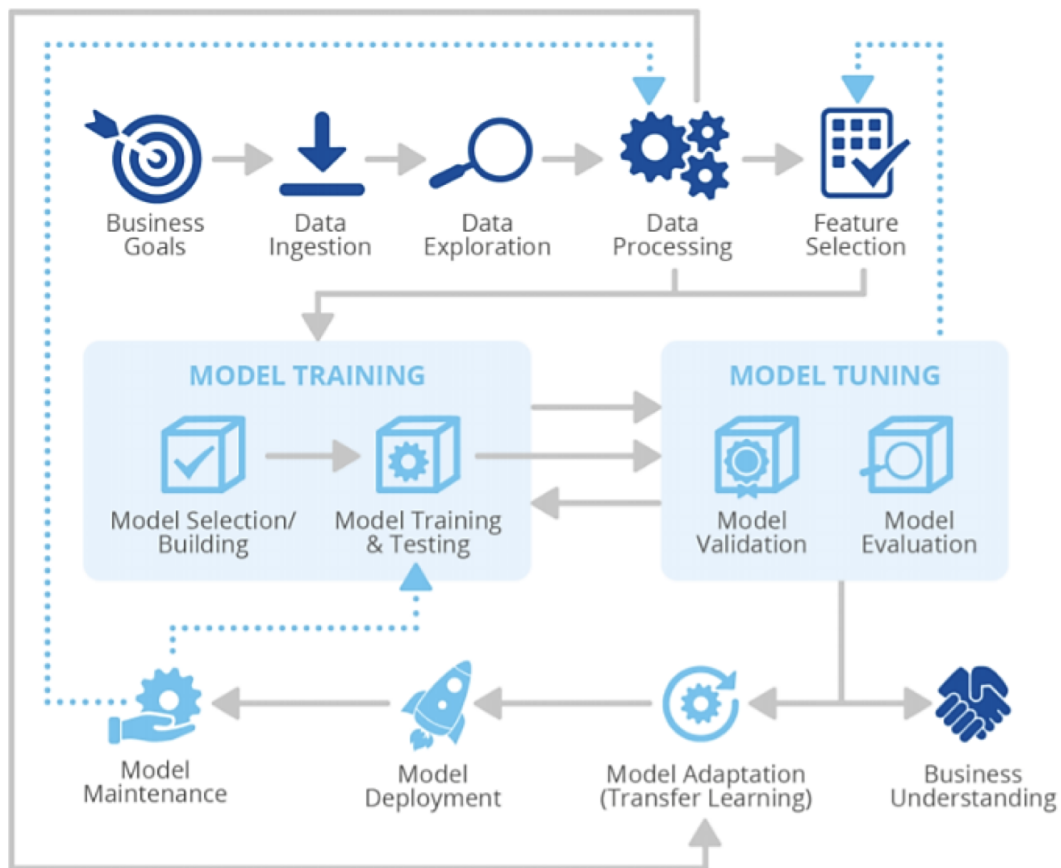


Figura 1.4: Ciclo di vita dell'AI e superficie di rischio SECoFAI. Fonte: [56].

Classi di minaccia e meccanismi. 1. **Evasion**: manipolazione degli input entro vincoli di norma (ℓ_p) per indurre decisioni errate in inferenza. 2. **Poisoning/backdoor**: contaminazione di dataset o pesi per introdurre comportamenti attivabili da trigger in produzione. 3. **Model extraction e abuso API**: replica funzionale via query e distillation; scraping di output, over-querying per denial. 4. **Attacchi alla privacy**: membership inference (presenza nel training set) e model inversion (ricostruzione di attributi dalle confidenze). 5. **Rischi LLM/GenAI**: prompt injection, insecure output handling, model DoS, tool/plugin non isolati, esfiltrazione in pipeline RAG. Tassonomie e linee guida comunitarie consolidano queste categorie e i controlli associati. [56, 57]

Supply chain AI. La catena di fornitura AI non è un singolo componente, ma una sequenza di risorse che vanno dai dataset ai modelli fondativi, dai pesi ai tokenizer, dai container ai tool MLOps. Una debolezza in uno di questi anelli può propagarsi fino all'ambiente di produzione. Per questo ogni

elemento necessita di provenienza verificabile, integrità attestata e criteri di aggiornamento sicuro documentati. [59]

1.4.3 Contromisure tecniche: robustezza, privacy, supply chain

Robustezza contro evasion e abusi. Questo insieme di controlli rende il modello stabile a input manipolati e riduce l'abuso delle API di inferenza (query massicce, scraping, estrazione modello) [58]. Tecniche principali: 1. **Adversarial training**: inserimento in addestramento di esempi perturbati entro un budget di norma per spostare il confine decisionale; trade-off su accuratezza e costi computazionali. 2. **Randomized smoothing**: rumore controllato all'input e voto maggioritario su più valutazioni; certifica un raggio di robustezza in cui l'etichetta resta invariata. 3. **Ensemble**: combinazione di modelli eterogenei (bagging o architetture diverse) per ridurre varianza e trasferibilità degli attacchi. 4. **Confidence calibration**: ritaratura delle probabilità (es. temperature scaling) per evitare overconfidence e migliorare triage/soglie. 5. **Indurimento dell'interfaccia**: rate limiting adattivo, proof-of-work, throttling per tenant, rilevamento di over-querying e blocchi progressivi.

Protezione dei dati e degli output. Questi controlli limitano leakage durante training/serving e impediscono riusi non sicuri a valle [59]. 1. **Data minimization e scoping**: raccolta del minimo necessario, data map e retention; separazione dei domini per ridurre la superficie d'attacco. 2. **Partizionamento per sensibilità**: storage e percorsi di calcolo segregati (pubblici, interni, sensibili), tokenizzazione e controlli di accesso differenziati. 3. **Federated learning**: addestramento con dati in sede e condivisione di gradienti/pesi; attenzione ai canali collaterali. 4. **Differential privacy**: DP-SGD e accounting di privacy (ϵ, δ) per ridurre membership/inversion, monitorando l'impatto sulle metriche. 5. **Output filtering e policy enforcement**: filtri sintattici/semantici sugli output e convalida prima di esecuzione/pubblicazione; per casi critici è raccomandato un human-in-the-loop. 6. **Audit sicuro**: logging mascherato e minimizzato, con controlli di accesso e retention coerenti con normative e rischio.

Sicurezza della supply chain AI. Si indirizzano provenienza, integrità e riproducibilità di dataset, modelli e pipeline MLOps lungo tutto il ciclo di vita, fino al rilascio e al serving [59]. 1. **Provenienza e attestazioni:** firme e checksum di dataset, pesi, tokenizer, container; attestazioni di build/training con registri centralizzati e stati di approvazione. 2. **Governance dei dataset:** hashing dei rilasci, controlli di licenza, allowlist delle fonti, canary samples anti-avvelenamento, separazione netta tra train/validation/serving. 3. **Build e training riproducibili:** container immutabili, dipendenze fissate, scansione CVE e principio del privilegio minimo; semi casuali e pipeline deterministiche per forensics affidabili. 4. **Promozione controllata:** passaggi tra ambienti con gate di integrità, prestazioni e sicurezza; strategie blue/green o canary con rollback esercitato. 5. **Sicurezza delle API di inferenza:** mTLS o equivalenti, gestione sicura dei segreti, chiavi per tenant, controllo egress e rate limiting coerente con criticità del servizio. 6. **SBOM/MBOM e tracciabilità:** distinta base di software e modelli/dataset per mappare dipendenze e impatti; supporto ad audit e richiamo selettivo in caso di compromissione.

1.5 Quadro normativo e linee guida operative

Il quadro europeo impone requisiti tecnici e organizzativi che incidono direttamente su come progettiamo autenticazione, gestione sessioni, API e logging. Il **Cyber Resilience Act (CRA)** rende obbligatori approcci secure-by-design e secure-by-default per prodotti software e dispositivi: gestione delle vulnerabilità, canali di aggiornamento sicuri, documentazione tecnica e controllo delle dipendenze lungo il ciclo di vita. Ciò si traduce, per i sistemi web, in inventari delle componenti (incluse librerie e SDK), politiche di patching, SBOM, pipeline di build riproducibili e validazioni di sicurezza prima del rilascio. [23] La **Direttiva NIS2** sposta l'attenzione sull'organizzazione: governance del rischio, continuità operativa, sicurezza della supply chain, monitoraggio e notifica degli incidenti. Per i servizi applicativi significa definire ruoli e responsabilità, adottare autenticazione forte per gli accessi privilegiati, proteggere le API esposte, registrare e correlare gli eventi di sicurezza e verificare periodicamente l'efficacia dei controlli con test e audit. [25]

Il **GDPR** impone misure tecniche e organizzative adeguate alla natura dei dati e al contesto del trattamento: protezione dei dati per impostazione predefinita, minimizzazione, cifratura/pseudonimizzazione, controllo degli accessi, registri di trattamento, valutazioni d'impatto e gestione dei data breach. Questi principi guidano scelte concrete nell'autenticazione (MFA e session binding), nella federazione (SSO con confini di scopo), nella conservazione dei log (necessità, proporzionalità, tempi di retention) e nella protezione del traffico (TLS moderno). [26]

Per il settore pubblico italiano, le **Linee guida AgID 2025** focalizzano l'implementazione: uso di **SPID/CIE** per l'identità, criteri per livelli di assurance, requisiti per sessioni e cookie (HttpOnly, Secure, SameSite), obbligo di TLS aggiornato, hardening delle intestazioni (CSP, HSTS), protezione delle API (autenticazione del client, rate limiting, controllo degli scope) e raccolta di telemetrie utili all'audit. L'allineamento con le **NIST SP 800-63-4** supporta la selezione coerente di AAL/IAL/FAL, l'adozione di fattori resistenti al phishing (WebAuthn/passkey) e la progettazione di step-up su operazioni sensibili. [54, 29]

1.5.1 Regolamentazione dell'AI

Gestione del rischio integrata nel SGSI. La gestione del rischio AI, per essere efficace e auditabile, deve innestarsi nel SGSI esistente con ruoli, metriche e processi dedicati. In pratica servono: inventario dei sistemi AI e loro scopi, classificazione per impatto, criteri di go/no-go, model cards con limiti d'uso, release gates, telemetrie su drift/abuse/fairness osservabile e un audit trail completo. Framework come NIST AI RMF 1.0 e lo standard ISO/IEC 42001 offrono controlli verificabili per l'integrazione. [60, 61]

Regolamentazione UE. Nel contesto europeo, il Regolamento (UE) 2024/1689 (AI Act) introduce requisiti su gestione del rischio, qualità dei dati, logging, trasparenza e post-market monitoring, con obblighi proporzionati al rischio e sanzioni correlate. L'allineamento anticipato riduce costi di adeguamento e ritardi autorizzativi. [62]

Capitolo 2

Sicurezza del Web

Questo capitolo analizza la sicurezza del *web* lungo l'intera catena tecnica: dal canale di trasporto al codice applicativo, fino alla gestione delle sessioni e alle tendenze emergenti. L'obiettivo è fornire criteri operativi e verificabili per progettare, configurare e monitorare servizi esposti su Internet.

La prima parte inquadra la sicurezza delle comunicazioni (HTTPS/TLS 1.3, HSTS, HTTP/3), con scelte di configurazione e errori ricorrenti. Segue una rassegna delle principali vulnerabilità delle applicazioni accompagnata da esempi minimi e schemi. Vengono poi trattate in modo specifico e tecnico, la gestione e il dirottamento delle sessioni: session fixation, sidejacking, XSS-based, MITM/AiTM, con contromisure e playbook di risposta. Chiudendo il capitolo con una panoramica dei trend attuali e delle relative implicazioni pratiche.

2.1 Web Security

Quando si parla di sicurezza informatica, il web è forse il terreno più esposto. Ormai ci passa dentro quasi tutto: dal pagamento delle bollette al controllo della cartella clinica, dall'accesso ai portali universitari fino agli acquisti online o alle operazioni bancarie. In poche parole, gran parte della nostra vita digitale dipende dal fatto che questi sistemi restino affidabili.

Il rovescio della medaglia è evidente: se un servizio web non è sicuro, i danni possono essere enormi, non si tratta solo di furti di dati. Un portale sanitario bloccato o un sito bancario fuori uso possono significare appuntamenti saltati, bonifici interrotti, stipendi in ritardo.

Proprio per questo la **sicurezza del web** non può più essere trattata come un dettaglio tecnico. È diventata un pilastro delle strategie di governi, aziende e persino delle università è possibile osservare organizzazioni come **OWASP** che lavorano continuamente per monitorare le minacce e pubblicano classifiche aggiornate delle vulnerabilità più comuni, che oggi rappresentano un punto di riferimento internazionale per chi sviluppa o gestisce applicazioni online[35].

Sicurezza delle comunicazioni. Alla base della sicurezza delle applicazioni web vi è il protocollo HTTPS che fornisce confidenzialità, integrità e autenticazione del server incapsulando HTTP in TLS. Con TLS 1.3 il canale è negoziato in meno round-trip e con primitive moderne, riducendo superfici d'attacco ereditate dalle versioni precedenti [37].

Nel handshake TLS 1.3 il client invia il ClientHello con l'elenco dei gruppi ellittici supportati e una key share per ECDHE. Il server risponde con ServerHello selezionando i parametri e completa lo scambio ECDHE. Le chiavi di traffico sono derivate con HKDF a partire dal segreto ECDHE e dall'hash del transcript, così ogni messaggio risulta legato crittograficamente all'intera negoziazione. La cifratura dei record applicativi usa cifrari AEAD (AES_GCM o ChaCha20_Poly1305); contatori di sequenza e nonce per record assicurano integrità e non ripetizione. L'autenticazione del server avviene tramite certificato X.509 e catena di fiducia. L'uso di chiavi effimere garantisce Perfect Forward Secrecy: la compromissione della chiave privata del server non consente di decifrare traffico precedente [37].

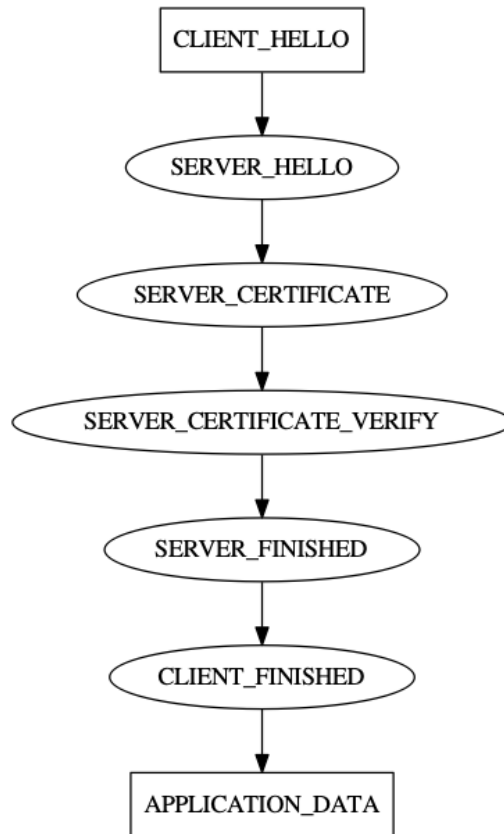


Figura 2.1: Schema sintetico di HTTPS/TLS 1.3: negoziazione della chiave (ECDHE), autenticazione del server e canale AEAD.

Riprese di sessione e 0-RTT. La ripresa può avvenire con PSK e session tickets emessi dal server; è buona pratica ruotare periodicamente le chiavi dei ticket. L'opzione 0-RTT riduce la latenza ma introduce rischio di replay: va disabilitata per operazioni non idempotenti o con effetti lato server [37].

Configurazione essenziale del server. 1. Abilitare TLS 1.3; mantenere TLS 1.2 solo se indispensabile e soltanto con cifrari AEAD. Disabilitare TLS 1.0/1.1 e gli scambi di chiave RSA statici; 2. Preferire ECDHE con firme ECDSA o RSA-PSS; usare chiavi almeno a 2048 bit (RSA) o curve P-256/P-384; 3. Impostare ALPN per HTTP/2 e/o HTTP/3; verificare la compatibilità con i client di destinazione; 4. Attivare OCSP stapling e monitorare l'intera catena di certificazione (incluse le CA intermedie); 5. Automatizzare emissione e rinnovo dei certificati con ACME; sorvegliare scadenze e SAN per evitare indisponibilità.

Protezione da downgrade ed errori di trasporto. Il solo redirect da HTTP a HTTPS è vulnerabile a SSL stripping. L'header HSTS obbliga il browser a usare sempre HTTPS e, con la preload list, elimina anche il primo contatto in chiaro [79].

```
Strict-Transport-Security: max-age=31536000; includeSubDomains;  
    preload
```

Per evitare contenuti misti si può applicare una CSP minimale:

```
Content-Security-Policy: upgrade-insecure-requests;  
    block-all-mixed-content
```

Errori operativi ricorrenti (da prevenire). 1. Certificati scaduti o con SAN errati: causano errori di validazione e interruzioni del servizio; 2. Versioni o cifrari deboli ancora abilitati: riaprono vettori noti (downgrade, assenza di PFS); 3. Mancanza di HSTS o preload: espone al downgrade sulla prima visita; 4. Mixed content su pagine HTTPS: vanifica la protezione del canale.

HTTP/3. HTTP/3 usa QUIC su UDP (porta 443) con TLS 1.3 integrato nel trasporto. Il handshake QUIC incorpora TLS 1.3, deriva subito le chiavi di traffico e cifra anche gran parte dei metadati del trasporto. Gli stream sono multiplexati a livello di trasporto e indipendenti, così si elimina l'head-of-line blocking tipico di HTTP/2 su TCP. Le connessioni sono identificate da Connection ID e possono migrare tra reti (ad esempio da Wi-Fi a rete mobile) senza interruzioni. L'early data 0-RTT è opzionale, basato su PSK/session tickets: riduce la latenza ma può essere soggetto a replay, quindi va disabilitato per metodi non idempotenti o operazioni con effetti lato server.

Dal punto di vista operativo dovremmo: annunciare ALPN h3 mantenendo il fallback h2; aprire UDP/443 su firewall e bilanciatori; curare MTU e Path MTU Discovery per evitare frammentazione; impostare timeouts e keepalive coerenti con l'ambiente. L'osservabilità richiede logging a livello applicativo o del reverse proxy, poiché QUIC cifra anche parte degli header; la terminazione può essere delegata a proxy/CDN con supporto HTTP/3, con fallback automatico ai client che parlano solo HTTP/2. Sul piano critto-

grafico restano valide le stesse scelte: ECDHE per la negoziazione e cifrari AEAD (AES_GCM o ChaCha20_Poly1305), con rotazione periodica delle chiavi tramite Key Update [37].

2.2 Vulnerabilità delle applicazioni web

Ogni volta che usiamo un sito o un servizio online potremmo dare per scontato che sia sicuro, ma in realtà dietro le quinte si nascondono errori ricorrenti che gli sviluppatori continuano a commettere. A ricordarcelo è l'**OWASP Top 10**, una classifica che viene aggiornata regolarmente e che fotografa i punti più deboli delle applicazioni web.

L'utilità dell'OWASP Top 10 non sta solo nell'elenco delle vulnerabilità, ma anche nelle raccomandazioni pratiche: suggerisce come evitare gli errori più comuni, quali strumenti usare in fase di sviluppo e quali controlli inserire nei test. Per questo motivo è diventato un riferimento sia nelle aziende private che nella pubblica amministrazione. Alcuni enti italiani lo citano addirittura nei capitolati tecnici dei bandi di gara, chiedendo che i software rispettino queste linee guida [35].

La superficie d'attacco di un'applicazione web si divide in due macro-aree: **client-side** (codice eseguito nel browser dell'utente) e **server-side** (codice e integrazioni eseguiti sul backend).

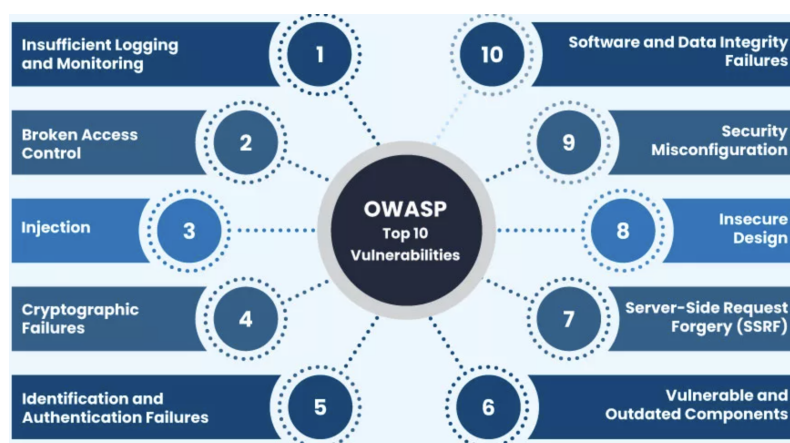


Figura 2.2: OWASP Top 10 Panoramica delle categorie di vulnerabilità più comuni nelle applicazioni web.

2.2.1 Client-side Security

Il browser esegue HTML, CSS e JavaScript, mantiene stato (cookie e Web Storage) e applica la same-origin policy. Le vulnerabilità client-side sfruttano l'esecuzione di script nel contesto della vittima, l'invio implicito di credenziali o l'inganno dell'interfaccia utente.

- **Cross-Site Scripting (XSS).** Consiste nell'esecuzione di JavaScript non voluto nel contesto della pagina della vittima. L'attacco nasce da output non correttamente codificato o da manipolazioni del DOM lato client alcuni degli mpatti tipici riguardano: furto di cookie di sessione non marcati HttpOnly, uso di API applicative tramite le credenziali dell'utente, defacement e keylogging. Le Varianti principali consistono: riflesso (input immediatamente riesposto), conservato (payload memorizzato su backend) e DOM-based (logica interamente nel browser) [3].

Esempio vulnerabile (DOM-based) e correzione essenziale.

```
<!-- Interpreta HTML inserito in query string -->
<div id="out"></div>
<script>
    const q = new URLSearchParams(location.search).get('q');
    document.getElementById('out').innerHTML = q;
</script>

<!-- Versione Corretto: tratta l'input come testo -->
<script>
    const q = new URLSearchParams(location.search).get('q');
    document.getElementById('out').textContent = q;
    // Aggiungere anche una CSP con nonce o hash
    // Content-Security-Policy: script-src 'self' 'nonce-<rand>';
</script>
```

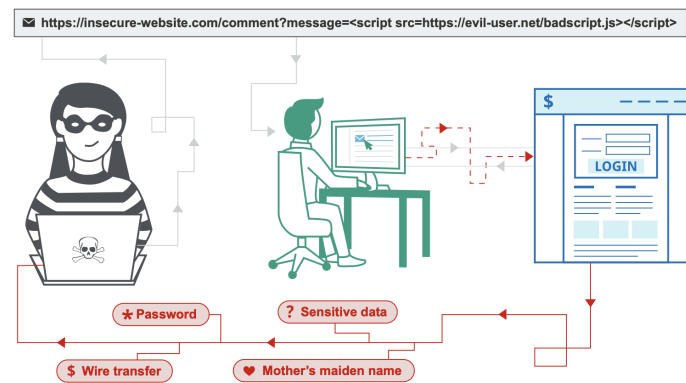


Figura 2.3: Schema XSS: riflesso, conservato e DOM-based.

- **Cross-Site Request Forgery (CSRF).** Una pagina terza induce il browser della vittima, già autenticata, a inviare richieste verso il sito bersaglio. Il browser allega automaticamente i cookie di sessione e il server può scambiare la richiesta per legittima, sfruttando così condizioni favorevoli: endpoint che accettano metodi modificativi senza token anti-CSRF, assenza di SameSite, CORS permissivo. Impatti: cambio email o password, azioni di conto, modifiche di configurazione [4].

Difesa con token anti-CSRF (synchronizer).

```

<!-- Form con token -->
<form action="/profile/email" method="POST">
  <input type="hidden" name="csrf" value="{{session.csrf}}">
  <input type="email" name="email">
  <button>Salva</button>
</form>

# Verifica lato server (pseudo)
assert timing_safe_equal(req.body.csrf, session.csrf)
# Impostare anche cookie SameSite=Lax/Strict
# e richiedere header non semplici sulle API.

```

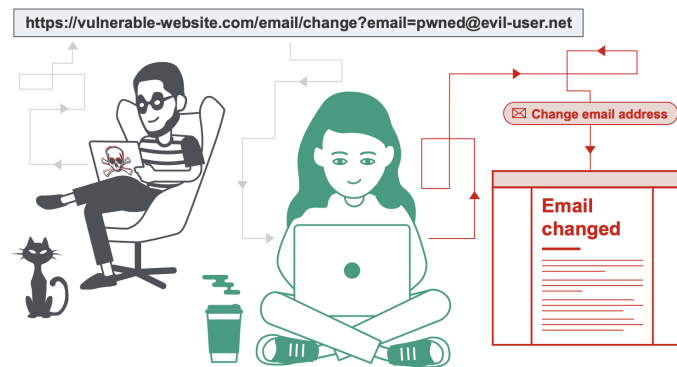


Figura 2.4: Schema CSRF: richiesta forgiata con cookie di sessione.

- **Clickjacking (UI redressing).** L'applicazione bersaglio viene caricata in un iframe trasparente o nascosto e sovrapposta a elementi grafici ingannevoli. L'utente clicca credendo di interagire con la pagina visibile, ma attiva controlli reali della pagina incorniciata facendo scaturire così: conferme di operazioni, abilitazione di funzioni, autorizzazioni. Difese efficaci: proibire l'inclusione in frame e limitare i domini che possono incorniciare l'app [5].

Esempio di attacco e intestazioni difensive.

```
<!-- Pagina attaccante: iframe sovrapposto e quasi invisibile
-->
<style>
  #trap { opacity:0.01; position:absolute; top:0;
    left:0; width:100%; height:100%; }
</style>
<iframe id="trap"
  src="https://bersaglio.xyz/account/delete"></iframe>

//Lato server (difesa):
X-Frame-Options: DENY
Content-Security-Policy: frame-ancestors 'none';
```

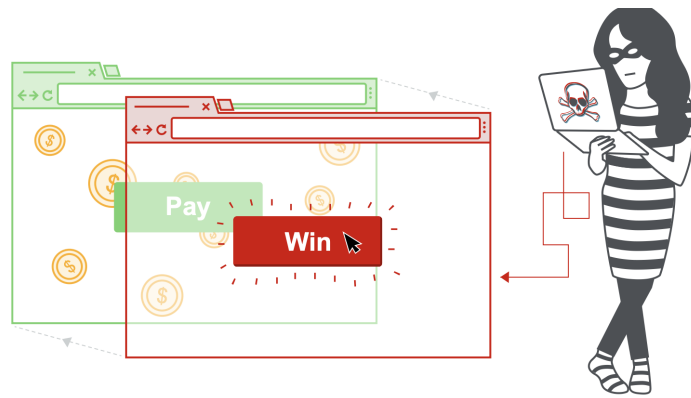


Figura 2.5: Tipologia di attacco clickjacking

- **CORS mal configurato (esfiltrazione cross-origin).** Il browser blocca per default la lettura di risposte cross-origin. Se il server espone `Access-Control-Allow-Origin` in modo errato, un sito terzo può leggere dati riservati. Errori tipici: riflettere indiscriminatamente l'header `Origin`, usare `*` insieme a `Access-Control-Allow-Credentials: true` (non valido ma spesso implementato in modo scorretto), includere tutta la wildcard di un dominio quando solo pochi sottodomini dovrebbero essere autorizzati. Impatti: lettura di API pensate per il primo dominio, leakage di dati utente [6].

Esempio di configurazione corretta.

```
// Corretto: allowlist esplicita e niente credenziali per
    origini generiche
const allow = new Set(["https://app.example",
    "https://admin.example"]);
if (allow.has(req.headers.origin)) {
    res.setHeader("Access-Control-Allow-Origin",
        req.headers.origin);
    res.setHeader("Access-Control-Allow-Credentials", "true");
    res.setHeader("Vary", "Origin");
} else {
    res.setHeader("Access-Control-Allow-Origin", "");
}
```

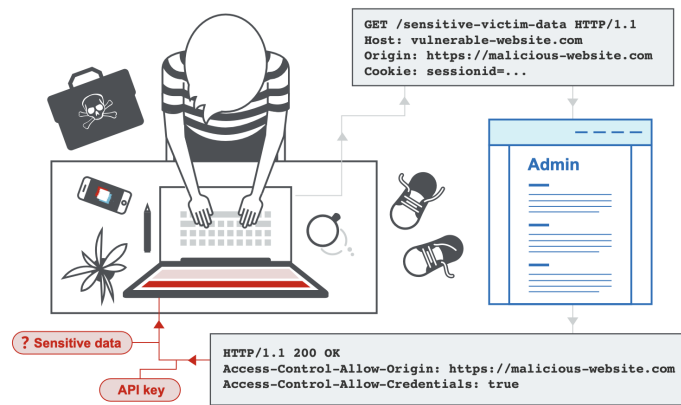


Figura 2.6: Tipologia di attacco Cors

2.2.2 Server-side Security

Il backend riceve richieste, valida input, compone query, chiama servizi di rete e può invocare processi locali. Le vulnerabilità più gravi nascono da input che raggiunge interpreti (SQL, shell) senza mediazione sicura, da fiducia implicita nella rete interna e da librerie che mascherano complessità ma non sostituiscono la validazione.

- **SQL Injection.** L'input dell'utente viene incorporato nella query SQL come testo, diventando metacomandi andando a comportare situazioni indesiderate come: lettura/scrittura arbitraria, bypass di login, estrazione tabelle, nel caso peggiore RCE tramite funzioni estese o procedure esterne. Esempi tipici: tautologie (' OR 1=1--), union-based, time-based (SLEEP), error-based.

Esempio vulnerabile e correzione.

```
// Vulnerabile: concatenazione
String q = "SELECT * FROM users WHERE email = '" + email +
    "'";
var row = db.query(q);

// Sicuro: statement parametrizzato
var row = db.query("SELECT * FROM users WHERE email = ?",
    [email]);
```

```
// Campo di ordinamento: whitelisting (niente placeholder
    per identificatori)
Set<String> allowed = Set.of("created_at","last_login");
String col = allowed.contains(sortBy) ? sortBy :
    "created_at";
String sql = "SELECT * FROM users ORDER BY " + col + " DESC
    LIMIT ? OFFSET ?";
```

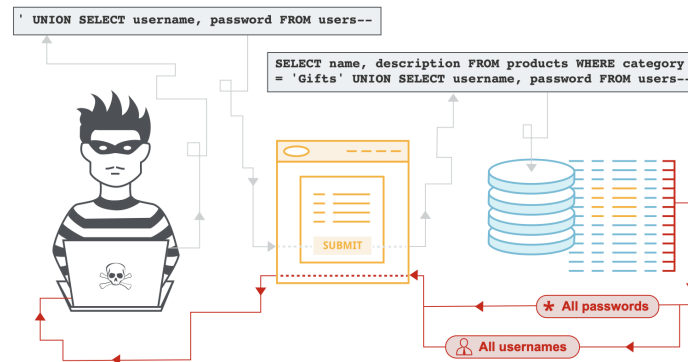


Figura 2.7: Schema SQL Injection: input non parametrizzato che altera la query.

- **Server-Side Request Forgery (SSRF).** L'applicazione effettua richieste HTTP/FTP/DNS verso URL forniti dall'utente; un attaccante può farle puntare a risorse interne non esposte (loopback, rete privata, metadata del cloud), sfruttando la posizione di rete privilegiata del server. Effetti principali: esfiltrazione di dati interni, scansione di servizi, acquisizione di credenziali temporanee da endpoint di metadata.

Esempio vulnerabile e correzione

```
// Vulnerabile: fetch su URL arbitrario senza controlli
const url = req.body.url; // es.
http://169.254.169.254/latest/meta-data/
const resp = await fetch(url);
const body = await resp.text();
```

```

// Corretto: allowlist, validazione post-DNS, egress
mediato da proxy

const parsed = new URL(req.body.url);
if (parsed.protocol !== "https:") deny();
const ip = await resolveAndPinIP(parsed.hostname);
if (isPrivateOrLinkLocal(ip)) deny();
const resp2 = await proxyFetch(parsed.toString(), {
  timeout: 3000, maxSize: 1<<20
});

```

- **OS Command Injection / RCE.** Si verifica quando input controllato dall'utente viene inserito in una stringa di comando o passato a un interprete di sistema (shell) tramite API come `system()`, `popen()`, `exec`, `child_process.exec` (Node), `Runtime.exec` (Java) o `subprocess(..., shell=True)` (Python). Poiché la shell interpreta metacaratteri (`;`, `&&`, `'...'`, `$()`, `>`, `<`, `*`, `&`, `newline`), espande variabili e wildcard, l'attaccante può chiudere l'argomento previsto e concatenare opzioni o comandi propri. L'effetto è esecuzione arbitraria con i privilegi del processo web (lettura/scrittura file, esfiltrazione, reverse shell, movimento laterale), tipicamente innescata da wrapper di utility esterne (`tar`, `convert`, `ffmpeg`) o da API che accettano un'unica stringa di comando. [8, 9, 10, 11]

Esempio vulnerabile e correzione (Node.js-like).

```

// Vulnerabile: costruisce una riga di shell
exec("convert " + filename + " -resize 200x200
thumb.png");

// Sicuro: API senza shell e parametri separati
spawnFile("convert", [filename, "-resize", "200x200",
"thumb.png"]);

```

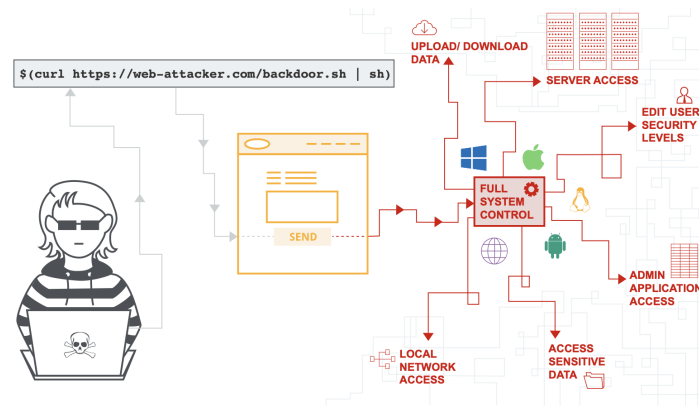


Figura 2.8: Schema OS Command Injection/RCE: input non validato passato alla shell.

2.3 Gestione e dirottamento delle sessioni

Ogni volta che ci autenticiamo su un sito, ad esempio inserendo utente e password, il sistema deve avere un modo per ricordarsi chi siamo finché restiamo collegati, proprio per evitare che durante l'utilizzo del sistema non ci venga richiesto di identificarci nuovamente. Per questo vengono usati i **cookie di sessione**, piccoli dati che il server assegna al browser e che viaggiano insieme a tutte le richieste successive.

Per migliorare la sicurezza di questo processo sono stati introdotti diversi accorgimenti. Il flag **HttpOnly**, ad esempio, impedisce a script eseguiti lato client di leggere il contenuto del cookie. L'opzione **Secure** garantisce che venga trasmesso solo su connessioni cifrate (HTTPS). Infine, l'attributo **SameSite** limita l'uso dei cookie nelle richieste provenienti da domini esterni, contribuendo a ridurre il rischio di attacchi come il **Cross-Site Request Forgery (CSRF)**. [47]

2.3.1 Vulnerabilità nella gestione delle sessioni

La sessione è ciò che permette a un sito di riconoscere un utente dopo il login e di mantenerne attiva la navigazione [65]. Senza questo meccanismo, ogni richiesta al server verrebbe vista come isolata e non legata all'utente che l'ha generata.

Il problema è che questa stessa funzionalità, indispensabile per far funzionare le applicazioni web moderne, può trasformarsi in un punto debole. I cookie e i token che identificano una sessione possono infatti essere sottratti con diverse tecniche: ad esempio catturando il traffico se non è cifrato, inserendo codice malevolo all'interno di una pagina (XSS), oppure forzando richieste costruite ad arte (CSRF).

Se un attaccante riesce ad avere in mano quel cookie, per il server diventa automaticamente l'utente legittimo: può entrare nell'account, leggere o modificare dati e muoversi liberamente, senza conoscere né username né password. Questo tipo di attacco prende il nome di **Session Hijacking** e viene trattato anche nelle linee guida OWASP [35], con numerosi esempi legati a portali bancari, servizi di e-commerce e piattaforme cloud.

In sintesi, l'accesso iniziale ci permette di entrare nel sistema, ma è la gestione sicura della sessione (spesso non sempre funzionante) che decide se quel collegamento resterà davvero sotto il nostro controllo.

Il session hijacking consiste nell'ottenere o riutilizzare l'identificatore di sessione di un utente legittimo per impersonarlo. I vettori principali si distinguono in quattro famiglie operative: 1. **Debolezze di gestione della sessione (server/app)**: Session Fixation; 2. **Compromissione del trasporto o della rete (network)**: Session Sidejacking, Man-in-the-Middle (MITM); 3. **Compromissione del client/applicazione (browser/app)**: Cross-Site Scripted Hijacking (XSS-based); 4. **Proxy malevolo tra utente e servizio (reverse proxy di phishing)**: Adversary-in-the-Middle (AiTM).

Session Fixation

Con il termine **Session Fixation** si indica una tecnica in cui l'attaccante convince la vittima a usare un identificatore di sessione che lui conosce già. In questo modo, quando la vittima effettua il login, l'aggressore ha la possibilità di entrare nello stesso account sfruttando quell'ID. Il problema si presenta soprattutto se l'applicazione non cambia automaticamente il codice di sessione dopo l'autenticazione. Analisi condotte nel 2024 su portali istituzionali europei hanno mostrato che circa il 15% risultava esposto a questa debolezza,

spesso per l'uso di URL rewriting o per una gestione superficiale dei cookie di sessione [65].

Dal punto di vista operativo, la fissazione avviene quando l'ID di sessione viene trasmesso o imposto alla vittima prima del login (per esempio in URL del tipo `/login;jsessionid=...` o `?sid=...`, tramite link o form precompilati, o mediante cookie impostati su canali non cifrati o da sottodomini con `Domain=.example.com`). Se l'applicazione non esegue la *session regeneration* al momento dell'autenticazione, l'ID prelogin resta valido dopo il login e diventa l'ID della sessione autenticata. L'aggressore, che conosce quell'ID, può presentarlo al servizio e ottenere l'accesso. Indicatori tipici sono la presenza del SID negli URL o nei referrer, log che mostrano lo stesso `session_id` prima e dopo l'evento di `login_success`, e l'assenza di un nuovo `Set-Cookie` postlogin.

Session Sidejacking

Il **Session Sidejacking** sfrutta l'intercettazione del traffico di rete non protetto. Se i cookie di sessione viaggiano in chiaro, basta essere collegati alla stessa rete per catturarli e impersonare l'utente. Uno dei casi più noti risale al 2010, con l'estensione *Firesheep* che intercettava cookie su Wi-Fi pubbliche [64]. Il problema persiste quando app o pagine servono ancora contenuti misti o non applicano rigorosamente TLS/HSTS [14].

Tecnicamente l'attacco consiste nello sniffing dei pacchetti che contengono `Set-Cookie` o richieste con cookie su HTTP non cifrato (anche tramite asset secondari in mixed content). L'assenza del flag `Secure` permette la fuoriuscita dei cookie su HTTP; l'assenza di HSTS abilita il downgrade/SSL stripping alla prima visita o dopo captive portal, costringendo il browser a iniziare in chiaro. Una volta acquisito il valore del cookie, l'aggressore lo riutilizza (*replay*) finché non scade o non viene ruotato dal server.

Cross-Site Scripted Hijacking (XSS-based)

Quando un sito è vulnerabile a XSS, l'attaccante può iniettare JavaScript che legge i cookie (se non `HttpOnly`) o usa direttamente i privilegi dell'utente per inviare richieste al server. Nelle applicazioni moderne l'impatto riguarda anche

token in `localStorage/sessionStorage`; se rubati, l'accesso resta valido a lungo. Con la diffusione delle SPA, l'effetto di un XSS è particolarmente critico [67].

Il payload XSS esegue codice nel contesto dell'origin della vittima e quindi eredita cookie, Web Storage e accesso al DOM. Da qui può: inviare chiamate `fetch/XHR` con `credentials: include` sfruttando la sessione della vittima; leggere token salvati nel Web Storage ed esfiltrarli; recuperare eventuali anti-CSRF token presenti nel DOM e riutilizzarli nelle richieste malevole. L'effetto pratico è il furto del token di sessione o il *session riding* delle API a nome dell'utente.

Man-in-the-Middle (MITM)

Il **Man-in-the-Middle** è “l'uomo in mezzo”: l'attaccante si interpone tra client e server, potendo leggere e/o modificare il traffico. Tecniche frequenti: ARP spoofing in LAN, DNS poisoning, access point Wi-Fi malevoli. TLS 1.3 rende lo sniffing molto più difficile, ma restano vettori come SSL stripping in assenza di HSTS o l'installazione di CA malevole sul device [37].

In assenza di HSTS, l'aggressore può effettuare SSL stripping bloccando o alterando i redirect a HTTPS e mantenendo la sessione in chiaro. In scenari di *TLS interception*, se il dispositivo accetta una CA non fidata (proxy aziendale configurato impropriamente o malware), il MITM termina TLS e ne ristabilisce un altro verso il server, leggendo e modificando le richieste. Nella fase di bootstrap di sessione questo consente di catturare o alterare **Set-Cookie**, parametri di federazione e header di autenticazione.

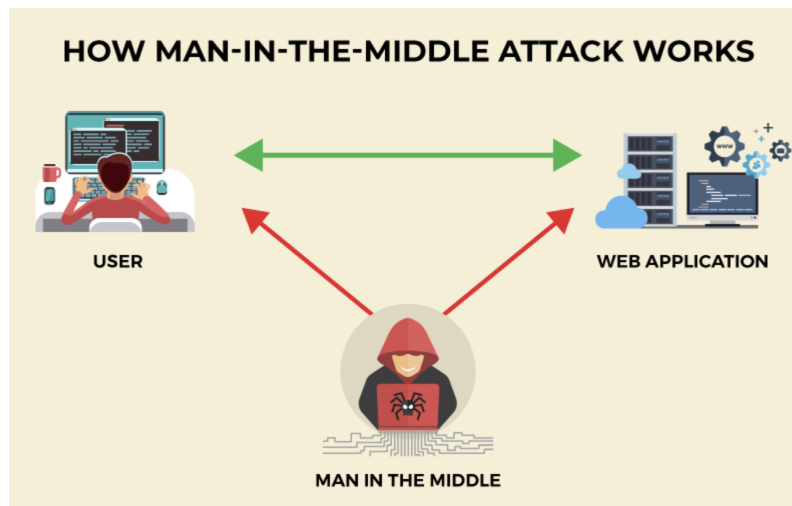


Figura 2.9: Schema MITM: l'attaccante si interpone tra client e applicazione web, intercettando o alterando il traffico.

Adversary-in-the-Middle (AiTM)

Con l'**Adversary-in-the-Middle** l'attaccante usa un reverse proxy che clona il sito legittimo e inoltra in tempo reale traffico e credenziali. Quando la vittima inserisce MFA e completa l'accesso, i **token di sessione** e di federazione vengono catturati dal proxy e riutilizzati dall'attaccante per entrare senza password né MFA [71].

Il proxy si pone tra browser e IdP/RP, riscrivendo dinamicamente i link e gli header e intercettando **Set-Cookie**, token **OIDC/SAML** e, quando presenti, header **Authorization: Bearer** usati dalle API. La MFA viene regolarmente validata dal sito reale, ma i token rilasciati *dopo* la MFA vengono copiati dall'aggressore e riutilizzati per instaurare una propria sessione. La persistenza è spesso ottenuta tramite refresh token e automazioni con browser headless per mantenere attive le sessioni.

2.3.2 Contromisure e Risposta agli incidenti

Se il trasporto è sicuro, il secondo fronte è l'applicazione e la gestione delle sessioni. I cookie di sessione vanno emessi con **HttpOnly**, **Secure** e **SameSite**; il prefisso **__Host-** vincola il cookie al dominio registrabile, richiede **Path=** e impedisce l'attributo **Domain** (niente riutilizzo da sottodomini), mentre **__Secure-** richiede almeno **Secure** e origine **HTTPS** [66]. Attenzione ai flussi federati:

quando un *front-channel* cross-site è necessario (SSO), i cookie coinvolti devono usare `SameSite=None; Secure`, altrimenti il flusso viene bloccato dal browser.

Per i token **JWT**, evitare lo storage nel Web Storage del browser (alto impatto in caso di XSS). Opzioni più robuste sono: conservarli in cookie protetti, oppure adottare un'architettura **Backend-for-Frontend** (Bff) che mantiene i token lato server e media tutte le chiamate dal client [67].

La gestione della sessione dovrebbe includere: (i) rigenerazione dell'ID dopo il login per azzerare ogni rischio di *session fixation* [66]; (ii) **token rolling** con rotazione periodica, così da ridurre la finestra di sfruttamento in caso di furto [66]; (iii) scadenze chiare (timeout di inattività e durata massima assoluta) e invalidazione lato server al logout [65, 66]. In ambienti ad alto rischio, è utile applicare controlli adattivi (re-auth/step-up su azioni sensibili) e segnali di anomalia sulla sessione (user-agent inconsistente, cambio repentino di ASN/geo, numero di IP per sessione), sempre con soglie compatibili con mobilità e privacy dell'utente.

Nel caso di un furto di sessione, la velocità di reazione è fondamentale: più tempo passa, più l'attaccante può muoversi liberamente nei sistemi. Alcuni campanelli d'allarme possono aiutare a individuare una compromissione in corso: accessi effettuati da Paesi diversi a distanza di pochi minuti, utilizzo di browser atipici (ad esempio i *headless browser* impiegati negli attacchi automatici), creazione improvvisa di regole nella posta elettronica per nascondere messaggi sospetti o collegamento di applicazioni OAuth mai autorizzate dall'utente [65].

Quando uno di questi segnali viene rilevato, l'organizzazione deve attivare un **playbook di risposta**, cioè una procedura prestabilita e collaudata che riduca al minimo i tempi di intervento. I passi più comuni includono: 1. revoca immediata di tutte le sessioni attive e generazione di nuovi token di accesso; 2. reset delle credenziali compromesse e, se possibile, della MFA, adottando soluzioni più robuste come FIDO2 per prevenire ulteriori abusi [32]; 3. controllo delle applicazioni OAuth collegate agli account e rimozione di quelle non autorizzate; 4. bonifica delle caselle di posta elettronica, eliminando regole malevole create dall'attaccante.

Una volta contenuto l'incidente, si passa alla fase di **analisi forense**, utile per capire come l'attacco sia avvenuto e quali dati siano stati esposti. Questa ricostruzione non serve solo a chiarire la dinamica, ma anche a migliorare i sistemi di difesa, aggiornando le regole di monitoraggio e i processi interni. In conclusione, ogni episodio di furto di sessione deve essere trattato come un'occasione per apprendere: l'esperienza maturata diventa parte integrante delle strategie di sicurezza future, riducendo il rischio che lo stesso scenario si ripeta [54].

2.4 Nuovi trend

Il panorama della web security evolve con architetture distribuite, automazione infrastrutturale e modelli di accesso moderni. Di seguito i trend più rilevanti per applicazioni e servizi esposti sul web, con implicazioni pratiche e controlli consigliati.

2.4.1 API Security di prima classe

Le applicazioni moderne espongono funzionalità via API (REST, GraphQL, gRPC) fra frontend, backend e terze parti. L'aumento del traffico machine-to-machine rende la superficie API un perimetro primario: errori di autorizzazione a livello oggetto (BOLA), validazioni carenti, assenza di rate limiting e gestione scorretta dei token sono oggi fra i principali driver d'incidente [44, 14]. La difesa efficace parte dal contratto: specifiche OpenAPI o schema GraphQL devono essere vincolanti a build e deploy, con validazione di input e risposta. L'autenticazione dei client esterni va realizzata con OAuth 2.0 e OIDC, verificando sistematicamente **issuer**, **audience**, firma e scadenze del token. L'autorizzazione deve essere minima e mirata: l'uso di **scope** granulari per funzione, claim dedicati per risorsa e controlli a livello di oggetto prevengono accessi orizzontali non intenzionali. È consigliabile ridurre la vita utile dei token, preferire token a prova di possesso (mTLS o DPoP) quando il rischio di riuso è alto e ruotare le chiavi di firma. A livello infrastrutturale un API gateway o una service mesh centralizzano verifiche JWT, applicano rate limiting, quota e circuit breaking, e impongono mTLS fra i workload. La telemetria deve inclu-

dere metodo, percorso, identità, risultato e tempi di risposta, con analisi delle anomalie per individuare abusi su endpoint sensibili [44]. Per GraphQL, oltre ai controlli precedenti, occorre limitare profondità e complessità delle query e disabilitare l'introspection in produzione.

2.4.2 Cloud-native e Kubernetes

Nei contesti cloud-native il modello perimetrale perde efficacia: container effimeri, orchestrazione e supply chain del software introducono nuovi punti di rottura [14, 23]. La protezione va impostata lungo tutta la catena. In fase di build si adottano immagini ridotte e firmate, si produce la SBOM e si blocca in pipeline ogni immagine con vulnerabilità note oltre una soglia. In ingresso al cluster, policy di ammissione impongono `runAsNonRoot`, file system in sola lettura, capacità minime, profili `seccomp` e `AppArmor`. La rete deve essere a default deny con `NetworkPolicy` che limitano traffico est e ovest; l'identità dei pod è gestita con account di servizio e ruoli cloud dedicati invece di credenziali statiche. I segreti non devono finire dentro l'immagine: si usano vault o KMS con cifratura e controllo accessi. A runtime servono controllo e allerta: audit del control plane, regole eBPF per intercettare comportamenti anomali (shell interattive nel pod, mount imprevisti, chiamate di rete atipiche), e immutabilità degli ambienti sensibili. Sul fronte normativo, il Cyber Resilience Act richiede gestione dei componenti e vulnerabilità lungo il ciclo di vita del prodotto, promuovendo pratiche di firma e tracciabilità della supply chain [23].

2.4.3 Zero Trust come modello operativo

L'adozione del modello Zero Trust sposta l'attenzione dalla rete "fidata" alla verifica continua: ogni richiesta è valutata in base a identità, contesto e stato del dispositivo [21]. Per i servizi web significa pubblicare le applicazioni dietro proxy consapevoli dell'identità, chiedere segnali di postura del device (integrità, patch, cifratura disco), e vincolare l'accesso a politiche espresse come codice. Le identità di utenti e workload sono ancorate a un IdP centrale (OIDC o SAML) con MFA resistente al phishing (FIDO2 o passkey) per ridurre l'impatto di AiTM. Le decisioni di accesso sono prese da un componente dedi-

cato (policy decision point) e fatte rispettare da un enforcement point vicino alla risorsa. I token devono essere di breve durata e legati al contesto; la segmentazione applicativa separa domini di fiducia interni e limita il movimento laterale. La telemetria completa su autenticazioni, decisioni di policy e uso delle applicazioni alimenta la rivalutazione continua del rischio e la risposta automatica a deviazioni [21].

2.4.4 Zero-day e Patching accelerato

Lo sfruttamento di vulnerabilità zero-day resta un acceleratore d'incidenti: exploit contro software lato client e componenti ausiliari (archiviatori, viewer, plugin) forniscono un trampolino verso gli account web tramite furto credenziali o di sessione. Nel 2025 una falla di WinRAR è stata usata in campagne mirate contro organizzazioni in Europa e Canada, consentendo esecuzione di codice e controllo remoto dei sistemi [46]. La mitigazione combina aggiornamento rapido, isolamento e igiene delle sessioni. I canali di update per client e agent devono essere tempestivi; quando il fix non è immediato si applica virtual patching su gateway web o WAF per ridurre l'esposizione degli endpoint. I client vanno ridotti all'essenziale, con sandbox delle applicazioni e disabilitazione dei formati non necessari; per allegati ad alto rischio è utile il content disarm and reconstruction. Sul piano identità, l'uso di FIDO2 o passkey limita l'efficacia del furto password; i token di sessione devono essere rotati, legati all'origine e revocati se un host mostra indicatori di compromissione. La risposta deve includere revoca mirata delle sessioni, reautenticazione delle operazioni sensibili e blocco degli indicatori nelle pipeline di posta e proxy [14, 46].

Capitolo 3

Ingegneria sociale e Phishing

Questo capitolo introduce l'ingegneria sociale e il phishing con un taglio pragmatico. Dopo una panoramica delle principali tecniche e delle leve psicologiche che le rendono efficaci, vengono discussi gli impatti organizzativi e le pratiche di resilienza. Si entra quindi nel dominio del phishing: evoluzione delle campagne, tipologie più diffuse, contromisure tecniche e processi di rilevamento e risposta. Nella seconda parte si presenta la componente sperimentale: progettazione e validazione, in ambiente controllato, di una landing di test mediante il framework JhonCecio, seguita dal posizionamento rispetto a strumenti di riferimento (SET e BlackEye). In chiusura, si introduce il tema del furto di sessione mediante reverse proxy (Evilginx) per evidenziare rischi e contromisure quando l'obiettivo non sono solo le credenziali ma la sessione applicativa.

3.1 Ingegneria Sociale

Parlando di sicurezza informatica, siamo abituati a immaginare muri digitali fatti di firewall, sistemi di cifratura o complesse procedure di login. Tuttavia, molto spesso non è la tecnologia a cedere per prima, ma le persone che la utilizzano ogni giorno. L'**ingegneria sociale** indica proprio questo: tecniche con cui un attaccante non forza direttamente un sistema, ma persuade un utente a consegnare ciò che lui richiede senza rendersene conto. Può trattarsi di una mail ben scritta che invita a cliccare su un link, di una telefonata in cui qualcuno si finge un tecnico del reparto IT o anche solo di un messaggio urgente che chiede di condividere una password [80, 83].

La ragione per cui queste tecniche funzionano è semplice: fanno leva su abitudini e reazioni umane comuni. La fiducia, la paura di perdere un'occasione, il desiderio di essere collaborativi o la fretta nel gestire le attività quotidiane diventano strumenti nelle mani dell'attaccante. Non serve violare un algoritmo complesso: basta spingere la vittima a compiere un piccolo gesto sbagliato. Ed è proprio questa capacità di sfruttare il fattore umano che rende l'ingegneria sociale tanto pericolosa quanto sottovalutata.

3.1.1 Tecniche di base

Pretexting Tra le tecniche più note c'è il **pretexting**, ovvero la costruzione di un falso pretesto convincente per ottenere informazioni. Un attaccante può presentarsi come un tecnico informatico che deve “verificare la rete”, oppure come un collega che ha “perso la password” e chiede assistenza. Se il contesto è costruito in modo credibile, molte persone forniscono spontaneamente le informazioni richieste.

Baiting Un'altra tecnica è il **baiting**, letteralmente “adescare”. Consiste nell'offrire qualcosa che attira la curiosità della vittima, come una chiavetta USB con un titolo conforme alle preferenze della vittima e lasciata in un ambiente da lui frequentato così facendo nel momento in cui il dispositivo viene collegato al computer, si attiva un malware.

Tailgating Molto diffuso anche il **tailgating**, cioè l'ingresso fisico non autorizzato in un edificio approfittando della cortesia di chi apre una porta senza controllare. È un esempio che dimostra come l'ingegneria sociale non si limiti al digitale, ma coinvolga anche la sfera fisica.

Quid pro quo Infine, il **quid pro quo** sfrutta il principio dello scambio: l'attaccante offre un apparente vantaggio, come assistenza gratuita o un software “premium”, chiedendo in cambio l'installazione di un programma che si rivela essere malevolo [35].

3.1.2 L'elemento psicologico

A differenza delle minacce puramente tecniche, l'ingegneria sociale punta tutto sulla psicologia. Le persone tendono a fidarsi delle figure autorevoli, a rispondere a richieste urgenti senza riflettere e a sottovalutare i rischi se si trovano in contesti familiari. Un'email che arriva da una persona presente nella vita della vittima che sia un amico o il proprio datore di lavoro avrà una percentuale di risposta più alta rispetto a un messaggio proveniente da uno sconosciuto con un messaggio generico.

Non è un caso che molti attacchi partano proprio da questi meccanismi. Nel 2023, ad esempio, diverse università europee hanno riportato casi di studenti e docenti che avevano consegnato le proprie credenziali a finti tecnici dell'assistenza, cadendo in trappole costruite su misura. Questo conferma che la vulnerabilità principale resta l'essere umano [65].

3.1.3 Effetti psicologici e reputazionali

Un attacco di ingegneria sociale non si misura solo in termini di dati rubati o sistemi bloccati. Spesso la parte più pesante è quella che non si vede: come reagiscono le persone coinvolte e che immagine resta all'esterno.

Chi cade in una trappola di phishing o fornisce senza volerlo informazioni sensibili non vive solo un errore tecnico. In certi casi la fiducia nelle proprie capacità cala, con ricadute sul lavoro quotidiano e sul clima all'interno dei team.

Dal lato dell'organizzazione, invece, entra in gioco la reputazione. Perdere la fiducia dei clienti è molto più difficile da rimediare che sostituire un server compromesso. Basta un episodio diffuso online perché la credibilità costruita in anni venga messa in discussione. E, a differenza di un sistema informatico, la reputazione non ha "patch" immediate: servono tempo, trasparenza e azioni concrete per recuperarla.

C'è poi l'aspetto normativo, come già espresso nel capitolo precedente il **GDPR**, ad esempio, obbliga enti e aziende a proteggere i dati personali: se questo non avviene, oltre al danno d'immagine arrivano anche multe e procedimenti uff-

ciali [26]. Ecco perché l'ingegneria sociale va vista non solo come un rischio tecnico, ma come un problema umano e di fiducia.

3.1.4 Resilienza umana contro l'ingegneria sociale

A differenza del phishing, dove le contromisure tecniche possono svolgere un ruolo importante, nell'ingegneria sociale la difesa principale resta la **consapevolezza delle persone**. Non basta avere sistemi sicuri se un dipendente, ingannato, fornisce di sua spontanea volontà password o informazioni riservate. Per questo motivo molte organizzazioni hanno introdotto programmi di **formazione continua**, che includono non solo lezioni teoriche, ma anche simulazioni di attacco. Ricevere una finta email di phishing, riconoscerla e segnalarla correttamente si è dimostrato molto più efficace che leggere linee guida astratte. Accanto alla formazione, sono cruciali procedure semplici e chiare: verificare sempre telefonicamente richieste anomale, evitare di aprire allegati da mittenti sconosciuti, segnalare comportamenti sospetti senza timore di conseguenze. L'obiettivo non è eliminare del tutto il rischio (cosa impossibile) ma ridurre la probabilità che un singolo errore umano comprometta l'intero sistema [35, 65].

3.2 Phishing

Il phishing resta una minaccia prevalente perché agisce sul fattore umano: induce l'utente a compiere un'azione rischiosa (clic su link, apertura allegato, inserimento credenziali) simulando comunicazioni legittime [76, 65]. La leva è psicologica (urgenza, autorità, curiosità), ma l'esecuzione è sempre più tecnica: domini "somiglienti", pagine clone su TLS valido, tracciamento delle vittime e infrastrutture *phishing-as-a-service* [83].

3.2.1 Origini ed evoluzione

Dalle prime campagne grossolane degli anni '90 si è passati a email curate, domini tipizzati (*typosquatting*) e omografi IDN (**xn--...**) che imitano visivamente i nomi originali. Con l'adozione massiva di servizi cloud e social, lo *spear phishing* è diventato mirato e contestualizzato; il *whaling* colpisce i

vertici per provocare transazioni o approvazioni fraudolente [80, 83]. Oggi il phishing è anche punto d'ingresso per intrusioni aziendali e ransomware [76].

3.2.2 Tipologie principali

1. **Spear phishing / whaling**: messaggi personalizzati con riferimenti reali a processi, colleghi o fornitori; spesso preparano *business email compromise* (BEC); 2. **Smishing e vishing**: SMS o telefonate che inducono all'azione (codici OTP, link a pagine clone); 3. **Clone phishing**: replica di email legittime con sostituzione di allegati/link verso risorse malevole; 4. **Consent phishing (OAuth)**: richiesta di consenso a un'app "terza" che ottiene token API (anche con MFA presente), permettendo accesso ai dati senza rubare password.

3.2.3 Impatti sulle organizzazioni

Le campagne moderne sono organizzate e multi-stadio. Credenziali o token compromessi portano a accessi interni, esfiltrazione e distribuzione di ransomware. Il BEC usa caselle reali compromesse per impartire istruzioni credibili, con perdite economiche significative documentate dai report annuali [72, 76].

3.2.4 Contromisure tecniche

La difesa è multilivello e combina autenticazione della posta, filtraggio avanzato e riduzione del valore delle credenziali.

Autenticazione della posta (SPF, DKIM, DMARC). SPF valida i server autorizzati a inviare per un dominio; DKIM firma crittograficamente i messaggi; DMARC allinea identità visibili e sottostanti (From, SPF, DKIM) e definisce la policy di trattamento. Una configurazione robusta prevede record SPF con meccanismi specifici (evitare `+all`), chiavi DKIM a 2048 bit con rotazione periodica e DMARC in `p=reject` con reportistica attiva (`rua`, `ruf`) [77, 78].

```
# Esempio di header utile al triage
Authentication-Results: mx.example; spf=fail
                        smtp.mailfrom=example.com
```

```
dkim=pass header.d=example.com; dmarc=quarantine action=quarantine  
header.from=example.com (alignment=fail)
```

MTA-STS e TLS reporting rafforzano il trasporto verso il dominio, mentre il blocco dei *lookalike domains* e la verifica dei record WHOIS aiutano a prevenire abusi d'identità.

Filtri, sandbox e protezioni lato utente. I gateway moderni eseguono analisi statiche e dinamiche su allegati (sandbox) e *time-of-click rewriting* per isolare link malevoli. Il blocco di macro, esecuzioni script nei documenti e HTML smuggling riduce il *payload risk*. Banner per mittenti esterni, disabilitazione di protocolli legacy (POP/IMAP basic), e regole anti-forwarding non autorizzato abbassano la superficie di abuso delle caselle.

Ridurre il valore di password e cookie. Autenticazione resistente al phishing (FIDO2/passkey), sessioni legate al contesto (device, rete, geografia) e token a breve vita riducono l'efficacia del furto credenziali o del riuso di sessione. Per l'accesso alle API da terze parti, preferire token a prova di possesso (mTLS o DPoP) e approcci *least privilege* con **scope** dedicati.

3.2.5 Rilevamento e risposta

Il rilevamento combina segnali lato posta e autenticazione: 1. anomalia nei risultati SPF/DKIM/DMARC, misalignment del **From** rispetto ai domini autorizzati; 2. pattern di invii in uscita atipici (BEC), regole di inoltro create di recente, risposte automatiche a catene esterne; 3. *impossible travel*, login da AS o Paesi insoliti, applicazioni OAuth appena autorizzate con permessi elevati.

La risposta deve includere: revoca e reset credenziali, invalidazione token/-sessioni, rimozione dei messaggi residui nelle caselle, notifica a utenti/partner, e *post-incident review* per correggere lacune (policy DMARC più restrittive, blocco protocolli legacy, rafforzamento MFA). Programmi periodici di simulazione e formazione restano essenziali per ridurre la probabilità di successo [54].

3.3 Progettazione e validazione sperimentale

Questa sezione descrive gli strumenti realizzati per la tesi con riferimento al phishing classico: dalla motivazione alla progettazione, fino all'implementazione e alla validazione in laboratorio di uno strumento leggero per simulare pagine di autenticazione verosimili. L'obiettivo è osservare in modo riproducibile il comportamento dell'utente e la risposta dei controlli difensivi quando l'interazione avviene su una landing di test che replica l'aspetto di servizi noti. Per supportare questo obiettivo è stato sviluppato JhonCecio, un framework minimale pensato per generare landing di laboratorio e raccogliere solo le evidenze necessarie alla valutazione difensiva, mantenendo separati punto di esposizione e backend. A fini di esercitazione e confronto sono stati impiegati – in ambiente di test – anche The Social-Engineer Toolkit (SET) e BlackEye. Per contestualizzare il contributo del lavoro rispetto alle soluzioni più diffuse sono state analizzate due piattaforme ampiamente impiegate nei programmi di awareness: GoPhish e King Phisher.

In estensione rispetto al phishing classico, verrà trattato e dimostrato successivamente anche il furto di sessione (session hijacking) mediante l'uso di Evilginx come reverse proxy per i domini di prova sono stati realizzati phishlet dedicati al solo scopo dimostrativo.

Tutte le attività sono state eseguite su infrastrutture di test con domini e account dedicati, in contesto autorizzato e confinato, nel rispetto delle linee guida [82]. Il codice implementato è disponibile nel repository GitHubLab.

3.3.1 JhonCecio: framework per simulazioni di phishing

JohnCecio è uno strumento leggero per simulare campagne di phishing tramite la clonazione dinamica degli endpoint di autenticazione di un sistema target, al fine di osservare il comportamento dell'utente. Il perimetro comprende: (i) clonazione controllata dell'interfaccia mediante template HTML su infrastruttura target, (ii) servizio locale con backend minimale in grado di ricevere le richieste della pagina simulata, (iii) esposizione opzionale verso l'esterno con separazione tra punto pubblico e backend, (iv) registrazione strutturata dei da-

ti inseriti nella simulazione con metadati essenziali, (v) reindirizzamento verso il dominio legittimo.

3.3.2 Confronto con Framework simili

Questa sezione presenta strumenti di riferimento comunemente impiegati in ambiente controllato per finalità didattiche e di ricerca. In generale, questi strumenti permettono di validare controlli tecnici e procedure in più contesti:

1. Email security: efficacia di SPF, DKIM, DMARC, regole antiphishing e URL rewriting del mail gateway nel bloccare o segnalare messaggi artefatti.
2. Browser e trasporto: comportamento degli indicatori di sicurezza del browser su domini non attesi; risposta di HSTS e di certificate pinning nei confronti di cloni in rete di test.
3. Identity e sessione: reazioni del flusso di autenticazione a parametri mancanti o alterati, robustezza di MFA phishing-resistant (FIDO2/passkey) contro pagine clonate in laboratorio.
4. Logging e risposta: qualità dei log lato server e proxy, correlazione alert nel SIEM, tempi di rilevamento e playbook di incident response in caso di inserimento dati nel clone simulato.

The Social-Engineer Toolkit (SET)

SET è un framework modulare per la simulazione controllata di attacchi di ingegneria sociale mantenuto da TrustedSec [74]. Opera come orchestratore di più tecniche, con particolare attenzione alla componente di consegna (email/SMS/telefono) e al canale web. In sintesi:

1. Architettura a moduli: componenti dedicati a spear phishing via email, pagine di raccolta credenziali (credential harvester), cloni controllati di form di login, e integrazioni con framework esterni. Ogni modulo espone opzioni e log separati.
2. Clonazione controllata: il modulo di clone ricostruisce una pagina di autenticazione target in ambiente di test, sostituendo l'azione del form con un endpoint locale di raccolta. Questo consente di osservare il comportamento dell'utente in modo tracciabile e confinato, senza toccare sistemi reali.
3. Telemetria e tracciamento: SET annota user agent, indirizzo IP di origine, timestamp e valore dei campi inviati alla pagina simulata. I dati sono consultabili in log strutturati, utili per l'analisi post-campagna e per misurare

i tassi di clic o di inserimento credenziali in un contesto di awareness. 4. Integrazione con pipeline difensive: i messaggi generati in laboratorio possono essere inviati a caselle di test o a mail gateway dedicati per validare regole anti-phishing, controlli SPF/DKIM/DMARC e link inspection, senza coinvolgere utenti finali.

BlackEye

BlackEye è una raccolta di template di pagine di autenticazione simulate con un backend minimale di raccolta. È concepito per dimostrare, in un ambiente isolato e autorizzato, quanto siano convincenti i cloni di interfacce note quando il dominio o l'URL non vengono verificati con attenzione. Aspetti tecnici rilevanti in ottica difensiva:

1. Template e flusso: ciascun template contiene HTML/CSS/JS che replica il look-and-feel del servizio legittimo. Il form invia le credenziali a un endpoint locale che le registra su file o stdout. Non è necessaria integrazione con servizi esterni: l'intero flusso avviene in laboratorio.
2. Esposizione controllata: la dimostrazione può rimanere confinata su rete privata di test; in scenari didattici si evita ogni forma di esposizione su Internet. L'obiettivo è mostrare l'importanza della verifica di dominio/URL e dei segnali del browser (barra degli indirizzi, certificato, indicatori di sicurezza).
3. Evidenze raccolte: oltre a username e password inseriti volontariamente nella simulazione, un recorder tipico annota user agent, IP sorgente e orario, a beneficio dell'analisi forense del test. Queste evidenze aiutano a calibrare filtri di mail gateway e policy di navigazione sicura.
4. Limiti deliberati: i template non devono essere usati contro sistemi reali o utenti inconsapevoli. In ambito formativo, il focus è sull'educazione dell'utente e sulla validazione dei controlli di rilevamento, non sull'ottenere credenziali.

```
(ciccioeraudo34@kaliCiccio) - [~/Desktop/Blackeye/blackeye-im]
$ ./blackeye.sh
:: Disclaimer: Developers assume no liability and are not ::
:: responsible for any misuse or damage caused by BlackEye. ::
:: Only use for educational purposes!! ::

:: BLACKEYE-IM! By @The-Burning ::

[01] Instagram [17] DropBox [33] eBay
[02] Facebook [18] Line [34] Amazon
[03] Snapchat [19] Shopify [35] iCloud
[04] Twitter [20] Messenger [36] Spotify
[05] Github [21] GitLab [37] Netflix
[06] Google [22] Twitch [38] Reddit
[07] Origin [23] MySpace [39] StackOverflow
[08] Yahoo [24] Badoo [40] Custom
[09] LinkedIn [25] VK
[10] Protonmail [26] Yandex
[11] Wordpress [27] devianART
[12] Microsoft [28] Wi-Fi
[13] IGFollower [29] PayPal
[14] Pinterest [30] Steam
[15] Apple ID [31] Tiktok
[16] Verizon [32] Playstation

[ Choose an option: ]-[~]
```

Figura 3.1: BlackEye: elenco dei template disponibili in ambiente di test.

```
[*] Waiting victim open the link ...

[*] IP Found!
jq: parse error: Invalid numeric literal at line 1, column 4
jq: parse error: Invalid numeric literal at line 1, column 4
jq: parse error: Invalid numeric literal at line 1, column 4
jq: parse error: Invalid numeric literal at line 1, column 4
jq: parse error: Invalid numeric literal at line 1, column 4
jq: parse error: Invalid numeric literal at line 1, column 4
[*] IPv6: 127.0.0.1
[*] User-Agent: User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
[*] Country:
[*] Region:
[*] City:
[*] Postal:
[*] Location: null
[*] Maps: https://www.google.com/maps/search/?api=1&query=,
[*] ISP: null
[*] Timezone: null
[*] Saved: github/saved.ip.txt

[*] Waiting credentials ...

[*] Credentials Found!
[*] Account: ceraudoprovatesi@gmail.com
[*] Password: P4ssw0rdS3gr3t4!7
[*] Saved: sites/github/saved.usernames.txt

(ciccioeraudo34@kaliCiccio) - [~/Desktop/Blackeye/blackeye-im]
$
```

Figura 3.2: Credenziali e utente rintracciato.

GoPhish

GoPhish è una piattaforma orientata a programmi di awareness con gestione di campagne e reportistica. Nel contesto di laboratorio: 1. gestione destinatari di test, template di messaggi e landing a scopo formativo; 2. metriche di engagement per campagna e per destinatario su aperture, clic e invii di form simulati; 3. flussi pensati per cicli di training e valutazioni periodiche.

King Phisher

King Phisher è uno strumento per campagne tecniche gestite da team di sicurezza con controllo fine degli asset di prova. Focalizzate su: 1. gestione

centralizzata di campagne email e landing, con telemetria dettagliata; 2. supporto a personalizzazioni e integrazioni per casi d'uso avanzati; 3. adatto a scenari tecnici dove è richiesta osservabilità estesa e tracciamento accurato.

Rispetto ai framework descritti, per lo più focalizzati su clonazione statica e semplice esposizione locale, JhonCecio è caratterizzato da: 1. orchestrazione unificata (`start_tool.sh`) con rilevamento dinamico dell'endpoint ngrok e patch del proxy; 2. *double-hop* di pubblicazione (tunnel pubblico → proxy → tunnel interno) per separare URL esposto e backend; 3. tracciamento in tempo reale delle acquisizioni e reindirizzamento trasparente al dominio reale; 4. struttura dei dati di output coerente per template e timestamp, utile a fini forensi e di reportistica.

3.3.3 Architettura e componenti

L'architettura di **JhonCecio** è composta da cinque componenti coordinati da uno script di orchestrazione (`start_tool.sh`) che disaccoppia esposizione pubblica, instradamento e logica applicativa.

- **Backend Flask** (`server.py`): eroga il template selezionato, gestisce la richiesta `POST /login`, registra le evidenze su file strutturati e restituisce un reindirizzamento 302 verso il dominio legittimo.
- **Tunnel interno** (opzionale, es. ngrok): espone in modo controllato la porta del backend e fornisce un endpoint non divulgato all'utente finale.
- **Proxy PHP** (`index.php`): punto di ingresso pubblico; inoltra metodo, percorso, header e corpo verso l'endpoint interno, restituendo al client la risposta del backend senza stato applicativo.
- **Server PHP locale** (porta es. :8000): ambiente di esecuzione del proxy.
- **Tunnel pubblico** (es. tunnelto): espone la porta del proxy tramite un URL pubblico stabile per scenari di test controllati.

Lo script `start_tool.sh` rileva i template disponibili, avvia sequenzialmente i processi (backend, tunnel interno se abilitato, proxy, tunnel pubblico), applica

la patch dell'endpoint interno nel proxy, monitora in tempo reale i file delle credenziali e centralizza i log.

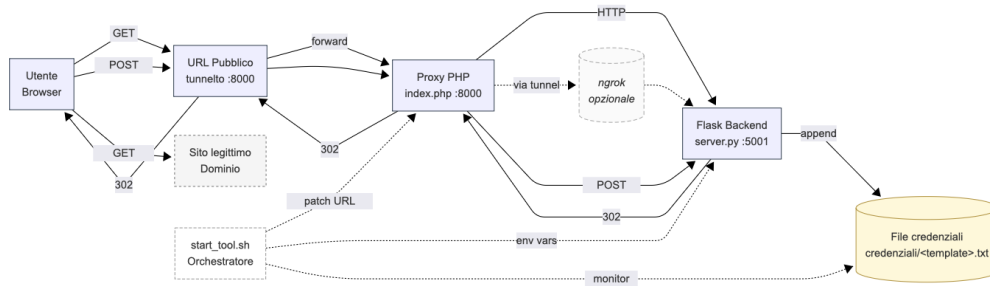


Figura 3.3: Architettura del framework JhonCecio

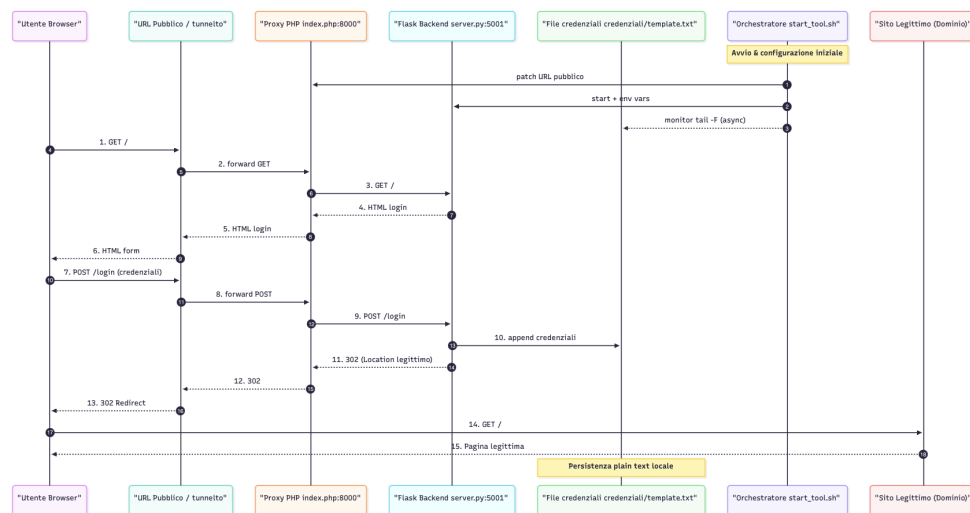


Figura 3.4: Diagramma di sequenza dell'architettura proposta: le richieste dell'utente sono instradate tramite l'URL pubblico verso il proxy PHP, inoltrate al backend Flask che serve la pagina di login, registra le credenziali su file locale e restituisce un redirect 302 verso il dominio legittimo.

Scelte progettuali e limiti

1. separazione dei livelli di esposizione. suddivisione tra URL pubblico, proxy intermedio e backend per ridurre l'esposizione diretta e permettere la sostituzione indipendente dei meccanismi di pubblicazione, persistenza su file strutturati con timestamp e identificatore del template; assenza di stato nel livello di ingresso;
2. template statici. riduzione delle dipendenze dal punto di vista del browser e assenza di problematiche di origin e CORS nei test;
3. telemetria

essenziale. raccolta dei soli metadati necessari alla validazione didattica e difensiva; 4. limiti. non include il canale di consegna dei messaggi, non affronta dinamiche di sessione complesse o aggiramento di fattori multipli; destinato esclusivamente a contesti autorizzati e isolati.

```
Numero scelta: 4
Scelta: amazon.html (site: amazon)

Avvio Flask server (server.py) sulla porta 5001

Avvio ngrok sulla porta 5001
[INFO] Wrote NGROK_URL=https://3a7a9dfe6707.ngrok-free.app into /Users/francescoceraudo/Desktop/Utils/PhishingUniversità/Tradizionale/index.php

Avvio PHP server su localhost:8000

Avvio tunnelto sulla porta 8000

Apri questo URL pubblico nel browser: https://t-bi4pb2gk.tunn.dev
```

Figura 3.5: Selezione template e generazione URL pubblico. Avvio dei servizi (Flask, tunnel, proxy) e stampa dell'URL di test.

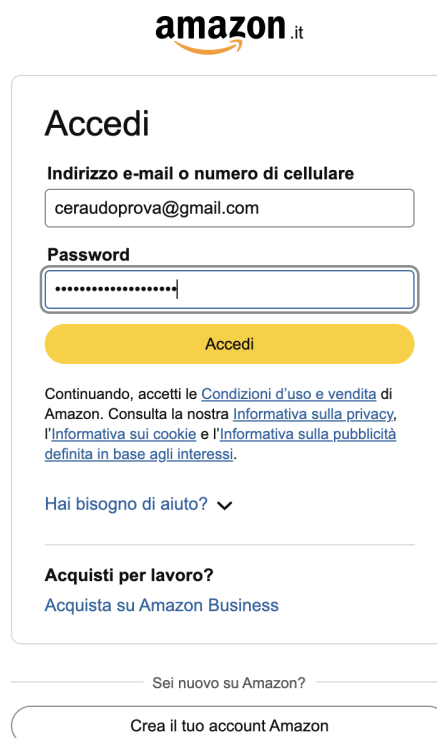


Figura 3.6: Login sul clone. Inserimento credenziali sulla pagina simulata servita da JhonCecio.

```

=====
Avvio PHP server su localhost:8000
=====
Avvio tunnelto sulla porta 8000
=====
Apri questo URL pubblico nel browser: https://t-bi4pb2gk.tunn.dev
Credenziali trovate
username: ceraudoprova@gmail.com
password: P4ssW0RdD1ff1c1l3!!!

```

Figura 3.7: **Acquisizione credenziali** Output della console con username e password registrati.

3.3.4 Reverse proxy per attacchi AiTM

Evilginx è un reverse proxy per scenari Adversary-in-the-Middle (AiTM) su flussi di autenticazione web [81]. Il browser instaura TLS verso il dominio di laboratorio; Evilginx, a sua volta, instaura TLS verso il sito reale, riscrivendo collegamenti e host nella risposta. Durante il bootstrap della sessione, le risposte del sito includono **Set-Cookie** (e, in alcuni casi, token in header o frammenti URL): Evilginx registra tali artefatti prima di inoltrarli al client, rendendo osservabile il rischio di riuso della sessione

Setup infrastrutturale: VPS, DNS e TLS

Per esporre in modo affidabile il dominio su porte 443/5300(scelta casuale) è stato utilizzato un server **VPS** (Virtual Private Server) in cloud, con IP pubblico statico e firewall configurato per consentire traffico in ingresso verso HTTP/HTTPS. La scelta della VPS è motivata da limiti comuni delle reti consumer: IP dinamici, NAT/captive e blocchi lato operatore che impediscono l'apertura di porte ben note. Sulla VPS sono stati configurati: 1. record DNS **A/AAAA** per il dominio di test puntati all'IP della VPS; 2. certificati TLS validi ; 3. reverse proxy Evilginx in ascolto su 80/443 con terminazione TLS server-side. Questa impostazione consente a Evilginx di presentare al browser un canale cifrato con certificato valido verso il dominio di test e, a valle, instaurare una seconda connessione TLS verso il sito reale.

Phishlet: definizione e sviluppo

In Evilginx il **phishlet** è il profilo che descrive come instradare e riscrivere un sito bersaglio: nomi host, percorsi, regole di sostituzione di URL e ri-

ferimenti a risorse statiche, normalizzazione di header e attributi dei cookie (Secure, HttpOnly, SameSite, Domain, Path). Per questa attività sono stati sviluppati due phishlet *proprietary*: uno per un flusso di autenticazione stile Git (GitHub/GitLab-like) e uno per Microsoft 365. Ogni phishlet specifica i domini/endpoint coinvolti nel login (IdP, static asset, API), le eccezioni di riscrittura e i pattern dei cookie osservati e catalogati a fini di laboratorio.

```

root@ubuntu-s-1vcpu-1gb-nyc3-01:~/evilginx2# ./evilginx2

[18:58:30] [inf] Evilginx Mastery Course: https://academy.breakdev.org/evilginx-mastery (learn how to create phishlets)
[18:58:30] [inf] loading phishlets from: /root/evilginx2/phishlets
[18:58:30] [inf] loading configuration from: /root/.evilginx
[18:58:30] [inf] blacklist: loaded 0 ip addresses and 0 ip masks
[18:58:30] [inf] obtaining and setting up 4 TLS certificates - please wait up to 60 seconds...
[18:58:30] [inf] successfully set up all TLS certificates

+-----+-----+-----+-----+-----+
| phishlet | status | visibility | hostname | unauth_url |
+-----+-----+-----+-----+-----+
| mgit     | enabled | visible   | progettocer.xyz |             |
| o365     | disabled | visible   | progettocer.xyz |             |
| poste    | disabled | visible   | progettocer.xyz |             |
+-----+-----+-----+-----+-----+

```

Figura 3.8: **Avvio del proxy AiTM.** Caricamento dei phishlet, ottenimento e installazione automatica dei certificati TLS, `mgit` attivo. Il proxy è pronto a terminare TLS lato laboratorio e ristabilire TLS verso il sito reale.

Phishlet Git: esito funzionante ed evidenze

Obiettivo: riprodurre via proxy l'interfaccia di autenticazione di un servizio Git con login web. Al termine del login, la risposta del sito reale ha rilasciato il cookie di sessione applicativa (es. `gh_session` o `_gitlab_session`), correttamente registrato dal proxy e riutilizzato per accesso *read-only* alle risorse dell'account di test.

phishlet	status	visibility	hostname	unauth_url
mgit	enabled	visible	progettocer.xyz	
o365	disabled	visible	progettocer.xyz	
poste	disabled	visible	progettocer.xyz	


```
: lures
```

id	phishlet	hostname	path	redirector	redirect_url	paused	og
0	mgit		/cUjUiazh				---
1	poste		/feUPAfBy				---
2	o365		/PoHtLVf				---


```
: lures get-url 1
```

<https://progettocer.xyz/feUPAfBy>

Figura 3.9: **Generazione del lure.** Interfaccia/testo di controllo su Evilginx con elenco dei *lures* e path univoci. Il comando `lures get-url` produce l’URL pubblico da usare nell’e-mail.

Messaggio fittizio (lure e-mail)

A scopo dimostrativo è stato generato un *lure* (URL di invito) da Evilginx e inserito in un messaggio e-mail **fittizio** destinato a un account di test . L’esempio seguente illustra struttura e posizionamento del link trappola (non destinato a invii reali).

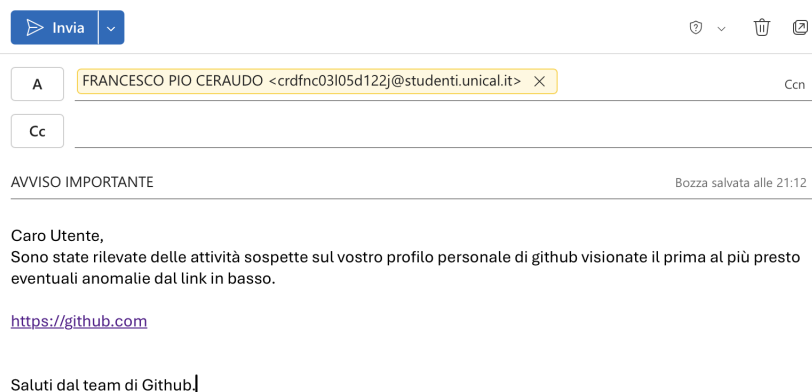


Figura 3.10: **E-mail di phishing** . Messaggio “security alert” che induce l’utente ad agire in urgenza e a verificare l’account GitHub tramite link. Il testo è plausibile e privo di allegati, tipico delle campagne che puntano al click.

Inserisci collegamento

Visualizza come

Indirizzo Web (URL)

OK Annulla

Figura 3.11: **Camuffamento del collegamento.** Il testo visibile è `https://github.com`, ma l'URL reale punta al lure generato dal proxy (`https://<dominio-di-lab>/<path>`). Classico mismatch “display vs. href” che sfrutta la fiducia visiva.

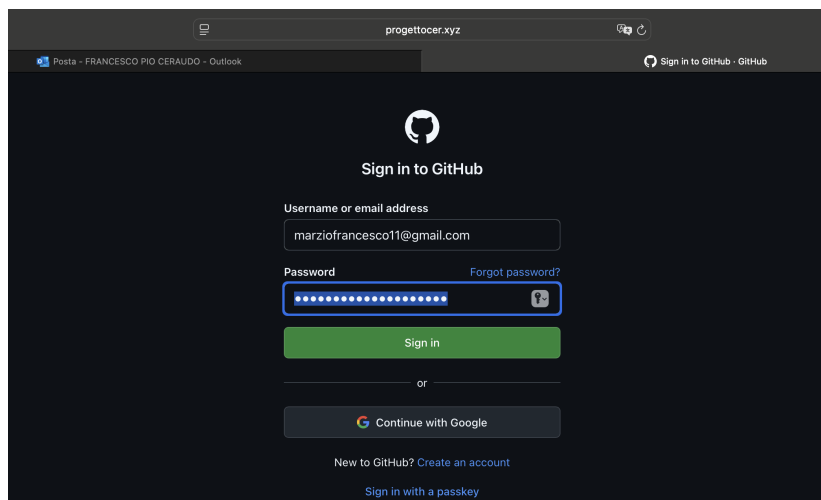


Figura 3.12: **Login proxato.** La pagina di GitHub viene resa tramite il dominio di laboratorio (certificato valido). L'utente inserisce le credenziali sul form caricato via proxy.

Phishlet Microsoft 365: esito non funzionante e analisi

Il tentativo analogo su Microsoft 365 non ha prodotto una sessione riutilizzabile. Il flusso coinvolge domini multipli e controlli addizionali a livello di identity provider e policy applicative: accesso condizionale (postura del device, rete/geo, rinnovo frequente dei token), protezioni anti-AiTM e telemetria che innesci nuova autenticazione o invalidazione selettiva su variazioni anomale [71]. In queste condizioni, un artefatto catturato fuori contesto perde valore operativo.

Confronto e lezioni apprese

Nel caso Git, la sessione post-login non era vincolata a segnali forti del dispositivo/contexto e il riuso in laboratorio è risultato possibile. Nel caso Microsoft 365, la combinazione di accesso condizionale, segnali di contesto e difese anti-AiTM ha impedito di trasformare la cattura in una sessione spendibile. Ne deriva che dove la sessione è un puro portatore di stato, un proxy AiTM può bastare; dove la sessione è ancorata a segnali ambientali e decisioni continue, la cattura tende a essere sterile.

Conclusioni

Questa tesi ha esplorato il panorama complesso della sicurezza informatica moderna, con particolare focus sulle vulnerabilità web e sulle tecniche di attacco basate su ingegneria sociale. Lo studio condotto mostra che la sicurezza IT non è la somma di misure isolate, ma un *sistema coerente* di scelte che richiede un approccio multidisciplinare per integrare aspetti tecnologici, normativi e umani al fine di affrontare le minacce contemporanee.

La disamina teorica ha confermato l'evoluzione del panorama delle minacce, dove l'intersezione tra intelligenza artificiale e sicurezza informatica rappresenta simultaneamente una risorsa difensiva e una nuova superficie di attacco. Il risultato è la necessità di approcci duali: AIxSEC per potenziare le difese e SECofAI per proteggere i sistemi basati su intelligenza artificiale.

L'approfondimento delle vulnerabilità web ha evidenziato come le applicazioni moderne richiedano una strategia difensiva stratificata che consideri tanto gli aspetti *client-side* quanto quelli *server-side*, mentre la gestione delle sessioni emerge come elemento critico, particolarmente vulnerabile a tecniche di dirottamento sempre più sofisticate che sfruttano tanto debolezze tecniche quanto fattori umani.

Il framework JhonCecio, sviluppato per questa tesi, rappresenta un contributo metodologico nell'ambito della simulazione controllata di attacchi phishing. I risultati degli esperimenti sugli attacchi di *session hijacking* condotti hanno fornito *insights* fondamentali sull'efficacia reale delle contromisure anti-AiTM. In particolare, nel caso Git, dove la sessione fungeva da semplice portatore di credenziali, gli attacchi AiTM si sono rivelati efficaci permettendo il riuso della sessione catturata, mentre nel caso Microsoft 365, l'integrazione di accesso condizionale, analisi dei segnali ambientali e difese specifiche anti-AiTM ha reso la cattura di sessione inefficace.

Questo lavoro dimostra che la sicurezza informatica moderna richiede una comprensione profonda dell'interazione tra tecnologia, psicologia e processi organizzativi, confermando come il fattore umano rimanga elemento centrale nella catena della sicurezza. La progressiva adozione di modelli Zero Trust e l'integrazione di intelligenza artificiale nei sistemi difensivi rappresentano direzioni promettenti per il futuro della sicurezza informatica.

Bibliografia

- [1] Samonas, S., & Coss, D. (2014). *The CIA Strikes Back: Redefining Confidentiality, Integrity and Availability in Security*. Disponibile su: <https://www.proso.com/dl/Samonas.pdf>
- [2] ENISA (2023). *Threat Landscape for DoS Attacks*. European Union Agency for Cybersecurity. Disponibile su: <https://www.enisa.europa.eu/sites/default/files/publications/DoS%20report.pdf>
- [3] OWASP Foundation (2023). *Cross-Site Scripting (XSS)*. Disponibile su: <https://owasp.org/www-community/attacks/xss/>
- [4] OWASP Foundation (2023). *Cross-Site Request Forgery (CSRF)*. Disponibile su: <https://owasp.org/www-community/attacks/csrf>
- [5] OWASP Foundation (2023). *Clickjacking*. Disponibile su: <https://owasp.org/www-community/attacks/Clickjacking>
- [6] OWASP Foundation (2023). *CORS Misconfiguration*. Disponibile su: <https://owasp.org/www-community/attacks/CORS-OriginHeaderScrutiny>
- [7] OWASP Foundation (2023). *Server-Side Request Forgery (SSRF)*. Disponibile su: https://owasp.org/www-community/attacks/Server-Side_Request_Forgery
- [8] OWASP Foundation (2023). *Command Injection*. Disponibile su: https://owasp.org/www-community/attacks/Command_Injection
- [9] MITRE (2024). *CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')*. Disponibile su: <https://cwe.mitre.org/data/definitions/78.html>

- [10] Node.js Foundation (2025). *child_process: exec, execFile, spawn — Security Considerations*. Documentazione ufficiale Node.js. Disponibile su: https://nodejs.org/api/child_process.html
- [11] Python Software Foundation (2025). *subprocess — Security Considerations*. Documentazione ufficiale Python. Disponibile su: <https://docs.python.org/3/library/subprocess.html#security-considerations>
- [12] IETF (2000). *RFC 2904: AAA Authorization Framework*. RFC Editor. Disponibile su: <https://www.rfc-editor.org/rfc/rfc2904>
- [13] Security Affairs (2022). *LockBit ransomware gang claims to have breached the Italian Revenue Agency*. Disponibile su: <https://securityaffairs.com/133640/cyber-crime/lockbit-ransomware-italian-revenue-agency.html>
- [14] ENISA (2024). *Threat Landscape Report 2024*. European Union Agency for Cybersecurity. Disponibile su: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2024>
- [15] SulPanaro.net (2025). *Cybersecurity, l'Emilia-Romagna al secondo posto per attacchi ransomware, visionata l'8 settembre*. Disponibile su: <https://www.sulpanaro.net/2025/06/cybersecurity-lemilia-romagna-al-secondo-posto-per-attacchi-ransomware/>
- [16] Przymus, P., & Durieux, T. (2025). *Wolves in the Repository: A Software Engineering Analysis of the XZ Utils Supply Chain Attack*. arXiv preprint arXiv:2504.17473. Disponibile su: <https://arxiv.org/abs/2504.17473>
- [17] Zhang, Z., Ning, H., Shi, F., Farha, F., Xu, Y., Xu, J., ... & Choo, K. K. R. (2022). *Artificial intelligence in cyber security: research advances, challenges, and opportunities*. Disponibile su: <https://link.springer.com/article/10.1007/s10462-021-09976-0>

- [18] IETF (2011). *RFC 6238: TOTP — Time-Based One-Time Password Algorithm*. Disponibile su: <https://datatracker.ietf.org/doc/html/rfc6238>
- [19] ANSI/INCITS (2012). *INCITS 359-2012: Role Based Access Control (RBAC)*. Disponibile su: <https://webstore.ansi.org/standards/incits/incits3592012>
- [20] NIST (2014). *SP 800-162: Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. Disponibile su: <https://csrc.nist.gov/publications/detail/sp/800-162/final>
- [21] Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). *Zero Trust Architecture (NIST SP 800-207)*. Disponibile su: <https://csrc.nist.gov/publications/detail/sp/800-207/final>
- [22] Ferrag, M. A., et al. (2024). *Generative AI in Cybersecurity: A Comprehensive Review of LLM Applications and Vulnerabilities*. arXiv preprint arXiv:2405.12750. Disponibile su: <https://arxiv.org/abs/2405.12750>
- [23] European Commission (2024). *Cyber Resilience Act*. Shaping Europe’s Digital Future. Disponibile su: <https://digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act>
- [24] Microsoft Entra (2023). *Microsoft Authenticator to enable number matching by default*. Disponibile su: <https://techcommunity.microsoft.com/t5/microsoft-entra-azure-ad-blog/microsoft-authenticator-to-enable-number-matching-by-default/ba-p/3716906>
- [25] European Union (2024). *Directive (EU) 2022/2555 on Measures for a High Common Level of Cybersecurity (NIS2)*. Disponibile su: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32022L2555>
- [26] European Data Protection Board (2024). *Annual Report 2024 on the Enforcement of the GDPR*. Disponibile su: <https://edpb.europa.eu>

- [27] Nguyen Ba, M. H., Bennett, J., Gallagher, M., & Bhunia, S. (2021). *A case study of credential stuffing attack: Canva data breach*. 2021 International Conference on Computational Science and Computational Intelligence (CSCI), IEEE. Disponibile su: <https://ieeexplore.ieee.org/document/9799087>
- [28] CERT-AgID (2021). *Analisi tecnica sull'attacco ransomware alla Regione Lazio*. Disponibile su: <https://www.altalex.com/documents/news/2024/05/07/attacco-hacker-sistemi-informatici-regione-lazio-intervento-garante-privacy>
- [29] NIST (2024). *Digital Identity Guidelines (NIST SP 800-63-4)*. National Institute of Standards and Technology. Disponibile su: <https://doi.org/10.6028/NIST.SP.800-63-4>
- [30] Rui, Z., & Yan, Z. (2018). *A survey on biometric authentication: Toward secure and privacy-preserving identification*. IEEE Access, 7, 5994–6009. Disponibile su: <https://ieeexplore.ieee.org/abstract/document/8590812>
- [31] AgID (2024). *SPID e CIE: Linee guida sull'identità digitale in Italia*. Agenzia per l'Italia Digitale. Disponibile su: <https://www.agid.gov.it>
- [32] Lyastani, S. G., Schilling, M., Neumayr, M., Schwenk, J., & Smith, M. (2020). *Is FIDO2 the kingslayer of user authentication? A comparative usability study of FIDO2 passwordless authentication*. Disponibile su: <https://ieeexplore.ieee.org/document/9152780>
- [33] Westers, M., Fett, D., Sommer, R., & Schwenk, J. (2023). *SSO-Monitor: Fully-Automatic Large-Scale Landscape, Security, and Privacy Analyses of Single Sign-On in the Wild*. Disponibile su: <https://arxiv.org/abs/2302.01024>
- [34] Podapati, V. H., Nigam, D., & Das, S. (2025). *SoK: A Systematic Review of Context- and Behavior-Aware Adaptive Authentication in Mobile Environments*. Disponibile su: <https://arxiv.org/abs/2507.21101>

- [35] OWASP Foundation (2021). *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. Disponibile su: <https://owasp.org/Top10>
- [36] Tadhani, J. R., Vekariya, V., Sorathiya, V., Alshathri, S., & El-Shafai, W. (2024). *Securing web applications against XSS and SQLi attacks using a novel deep learning approach*. Disponibile su: <https://doi.org/10.1038/s41598-023-48845-4>
- [37] IETF TLS Working Group(2018). *TLS 1.3—Updates to the most important security protocol on the Internet*. IETF blog. Disponibile su: <https://www.ietf.org/blog/tls13/>
- [38] Habib, M. I., Al Maruf, A., & Ahmed Nabil, M. J. (2023). *An Exploration Into Web Session Security – A Systematic Literature Review*. Disponibile su: <https://arxiv.org/abs/2310.10687>
- [39] Arghire, I. (2024). Millions of User Records Stolen From 65 Websites via SQL Injection Attacks. Disponibile su: <https://www.securityweek.com/millions-of-user-records-stolen-from-65-websites-via-sql-injection-attacks/>
- [40] Toulas, B. (2024). Hackers Steal Data of 2 Million in SQL injection, XSS Attacks. Disponibile su: <https://www.bleepingcomputer.com/news/security/hackers-steal-data-of-2-million-in-sql-injection-xss-attacks/>
- [41] OWASP Foundation (s.d.). *Session Fixation*. Disponibile su: https://owasp.org/www-community/attacks/Session_fixation
- [42] Cloudflare (s.d.). *What is a Man-in-the-Middle (MITM) attack?* Disponibile su: <https://www.cloudflare.com/learning/security/threats/man-in-the-middle-attack/>
- [43] U.S. Department of Health and Human Services, Office for Civil Rights (OCR).(più report presenti dei vari anni) *Annual Report to Congress on Breaches of Unsecured Protected Health Information*.

- Disponibile su: <https://www.hhs.gov/hipaa/for-professionals/breach-notification/reports-congress/index.html>
- [44] Chandramouli, R., et al. (2025). *Guidelines for API Protection for Cloud-Native Systems*. Disponibile su: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-228.pdf>
- [45] Università degli Studi di Milano - LASER Lab. (2022). *MEIEC Online - Penetration Testing Report*. Disponibile su: https://air.unimi.it/retrieve/63230e21-b6f2-41d7-900f-4fe0dbb7d9de/MEIEC_online-DEFINITIVO2.pdf
- [46] Kovacs, E. (2025). Russian Hackers Exploited WinRAR Zero-Day in Attacks on Europe, Canada. Disponibile su: <https://www.securityweek.com/russian-hackers-exploited-winrar-zero-day-in-attacks-on-europe-canada/>
- [47] Przymus, P., & Durieux, T. (2024). *Towards Browser Controls to Protect Cookies from Malicious Extensions*. arXiv preprint arXiv:2405.06830. Disponibile su: <https://arxiv.org/abs/2405.06830>
- [48] PwC (2021). *Post Incident Review into the Conti Ransomware Cyber Attack on the HSE (Health Service Executive)*. Report commissioned by the HSE, Dublin, December 3, 2021. Disponibile su: <https://www.hse.ie/eng/services/publications/conti-cyber-attack-on-the-hse-full-report.pdf>
- [49] W3C (2025). *Web Authentication: An API for accessing Public Key Credentials — Level 3*. Disponibile su: <https://www.w3.org/TR/webauthn-3/>
- [50] FIDO Alliance (2023). *Client to Authenticator Protocol (CTAP) v2.2*. Disponibile su: <https://fidoalliance.org/specs/fido-v2.2-rd-20230321/fido-client-to-authenticator-protocol-v2.2-rd-20230321.pdf>

- [51] NIST (2025). *SP 800-63C-4 — Digital Identity Guidelines: Federation and Assertions*. Disponibile su: <https://doi.org/10.6028/NIST.SP.800-63C-4>
- [52] FIDO Alliance (2022). *White Paper: Multi-Device FIDO Credentials*. Disponibile su: <https://fidoalliance.org/white-paper-multi-device-fido-credentials/>
- [53] FIDO Alliance (2024). *Passkeys: Passwordless Authentication*. Disponibile su: <https://fidoalliance.org/passkeys/>
- [54] AgID (2025). *Linee guida per la sicurezza dei servizi digitali in Italia*. Agenzia per l'Italia Digitale. Disponibile su: <https://www.agid.gov.it>
- [55] Du, M.; Li, F.; Zheng, G.; Srikumar, V. (2017). *DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning*. Proceedings of ACM CCS 2017. Disponibile su: <https://dl.acm.org/doi/10.1145/3133956.3134015>
- [56] ENISA (2021). *Securing Machine Learning Algorithms*. European Union Agency for Cybersecurity, dicembre 2021. Disponibile su: <https://www.enisa.europa.eu/publications/securing-machine-learning-algorithms>
- [57] OWASP (2025). *Top 10 for LLM and GenAI Applications (versione 2025)*. Disponibile su: <https://genai.owasp.org/llm-top-10/>
- [58] Goodfellow, I. J.; Shlens, J.; Szegedy, C. (2015). *Explaining and Harnessing Adversarial Examples*. arXiv:1412.6572. Disponibile su: <https://arxiv.org/pdf/1412.6572>
- [59] ISO/IEC (2023). *ISO/IEC 23894:2023 — Artificial intelligence — Guidance on risk management*. International Organization for Standardization. Disponibile su: <https://www.iso.org/standard/77304.html>
- [60] NIST (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. National Institute of Standards and Technology, Gaithersburg, MD

- sburg, MD. Disponibile su: <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>
- [61] ISO/IEC (2023). *ISO/IEC 42001:2023 — Artificial intelligence management system (AIMS) — Requirements*. International Organization for Standardization. Disponibile su: <https://www.iso.org/standard/42001>
 - [62] Unione Europea (2024). *Regolamento (UE) 2024/1689 che stabilisce norme armonizzate sull'intelligenza artificiale (AI Act)*. Gazzetta Ufficiale dell'Unione Europea, 12 luglio 2024. Disponibile su: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
 - [63] MITRE (2024). *AI Red Teaming: Advancing Safe and Secure AI Systems*. Priority Topic Memo, luglio 2024. Disponibile su: <https://www.mitre.org/sites/default/files/2024-07/PR-24-01820-4-AI-Red-Teaming-Advancing-Safe-Secure-AI-Systems.pdf>
 - [64] Butler, E. (2010). *Firesheep*(Blog) Disponibile su: <https://codebutler.com/2010/10/24/firesheep/>
 - [65] Habib, M. I., Al Maruf, A., & Ahmed Nabil, M. J. (2023). *An Exploration Into Web Session Security – A Systematic Literature Review*. Disponibile su: <https://arxiv.org/abs/2310.10687>
 - [66] OWASP Foundation (2023). *Session Management Cheat Sheet*. Disponibile su: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
 - [67] Tadhani, J. R., Vekariya, V., Sorathiya, V., Alshathri, S., & El-Shafai, W. (2024). *Securing web applications against XSS and SQLi attacks using a novel deep learning approach*. Disponibile su: <https://doi.org/10.1038/s41598-023-48845-4>
 - [68] IETF TLS Working Group. (2018). *TLS 1.3—Updates to the most important security protocol on the Internet*. IETF blog. Disponibile su: <https://www.ietf.org/blog/tls13/>

- [69] Ometov, A., Bezzateev, S., Mäkitalo, N., Andreev, S., Mikkonen, T., & Koucheryavy, Y. (2018). *Multi-Factor Authentication: A Survey*. Cryptography, 2(1), 1. Disponibile su: <https://doi.org/10.3390/cryptography2010001>
- [70] Hallam-Baker, P., Stradling, R. (2019). *RFC 8659: DNS Certification Authority Authorization (CAA) Resource Record*. IETF. Disponibile su: <https://www.rfc-editor.org/rfc/rfc8659>
- [71] Microsoft Entra Team (2024). *Defeating Adversary-in-the-Middle phishing attacks*. Disponibile su: <https://techcommunity.microsoft.com/blog/microsoft-entra-blog/defeating-adversary-in-the-middle-phishing-attacks/1751777>
- [72] FBI Internet Crime Complaint Center (IC3) (2024). *2023 Internet Crime Report*. Disponibile su: https://www.ic3.gov/Media/PDF/AnnualReport/2023_IC3Report.pdf
- [73] BBC News (2020). *Twitter Hack: Attackers Gained Access by Phishing Employees*. Disponibile su: <https://www.bbc.com/news/technology-53425822>
- [74] TrustedSec (2025). *The Social-Engineer Toolkit (SET)*. Disponibile su: <https://www.trustedsec.com/tools/the-social-engineer-toolkit-set/>
- [75] NIST (2008). SP 800-115: Technical Guide to Information Security Testing and Assessment. Disponibile su: <https://csrc.nist.gov/publications/detail/sp/800-115/final>
- [76] Verizon (2024). *Data Breach Investigations Report 2024*. Disponibile su: <https://www.verizon.com/business/resources/reports/dbir/>
- [77] Crocker, D., Hansen, T., & Kucherawy, M. (2011). *DomainKeys Identified Mail (DKIM) Signatures*. RFC 6376, IETF. Disponibile su: <https://www.rfc-editor.org/rfc/rfc6376>

- [78] Kucherawy, M., & Zwicky, E. (2015). *Domain-based Message Authentication, Reporting, and Conformance (DMARC)*. RFC 7489, IETF. Disponibile su: <https://www.rfc-editor.org/rfc/rfc7489>
- [79] Hodges, J.; Jackson, C.; Barth, A. (2012). *HTTP Strict Transport Security (HSTS)*. RFC 6797, IETF. Disponibile su: <https://www.rfc-editor.org/rfc/rfc6797>
- [80] Jagatic, T. N., Johnson, N. A., Jakobsson, M., & Menczer, F. (2007). *Social phishing*. Communications of the ACM, 50(10), 94–100. Disponibile su: <https://doi.org/10.1145/1290958.1290968>
- [81] Gretzky, K. (2025). *Evilginx2: reverse proxy for researching AiTM phishing*. Disponibile su: <https://github.com/kgretzky/evilginx2>
- [82] NIST (2008). *SP 800-115: Technical Guide to Information Security Testing and Assessment*. National Institute of Standards and Technology. Disponibile su: <https://csrc.nist.gov/publications/detail/sp/800-115/final>
- [83] Hong, J. (2012). *The state of phishing attacks*. Communications of the ACM, 55(1), 74–81. Disponibile su: <https://doi.org/10.1145/2063176.2063197>

Ringraziamenti

A mamma e papà. Devo più di quanto le parole possano dire, perché da solo potrei fare tanto, ma con voi ho imparato a dare valore a ciò che conta davvero. Grazie per avermi permesso di raggiungere questo traguardo con libertà e fiducia, senza pressioni né aspettative, ma con la certezza costante del vostro amore, per avermi lasciato lo spazio di sbagliare e di diventare me stesso. Dietro ogni passo importante della mia vita ci siete voi: la sicurezza che mi avete trasmesso, l'ascolto che non mi è mai mancato e la serenità di sapere che, qualunque strada scegliessi, avreste camminato accanto a me.

A mia Nonna Pasina. Una delle persone più importanti della mia vita. Ti sarò sempre riconoscente per aver avuto il privilegio di crescere con te, con la tua presenza rassicurante e quell'affetto che hai saputo dimostrare in ogni gesto e in ogni parola. Ogni mio traguardo, ogni esame, è sempre stato anche un po' tuo, perché hai vissuto con me ogni percorso, con orgoglio e con quella gioia sincera che sai trasmettere.

A mia Nonna Franca e a mio Nonno Peppe. Per la presenza costante, per l'affetto semplice e sincero con cui mi avete sempre circondato. Per i valori che mi avete trasmesso, per le storie, i gesti e le attenzioni che hanno reso la mia infanzia, la mia adolescenza e la mia giovane età un luogo sicuro e pieno di calore. Grazie per essere stati, e continuare ad essere, una parte fondamentale della mia vita.

A mio Zio Giuseppe. È difficile spiegarti ciò che sei per me con semplici parole. Ti ammiro per la persona sincera, leale e presente che sei, e ti ringrazio per esserci sempre stato, soprattutto nei momenti più difficili, senza che io dovessi mai chiedere nulla. La tua vicinanza, il tuo modo silenzioso ma costante di esserci, hanno rappresentato per me un punto di forza e di conforto. Tu, come papà, mi avete mostrato la capacità di mettere il bene degli altri prima del vostro. Vi ringrazio per avermi insegnato, con l'esempio, cosa significhi volersi bene davvero e quanto valore ci sia nell'essere presenti l'uno per l'altro. Crescendo, ho capito che la vita ci mette spesso davanti a ostacoli, grandi o

piccoli che siano, ma che, se affrontati insieme, anche le difficoltà più grandi diventano più leggere.

Alla mia famiglia. *Zii, zie e cugini, grazie per l'affetto sincero, per i momenti felici condivisi e per la forza con cui sappiamo restare uniti anche nelle difficoltà. In ogni risata, in ogni gesto, c'è il valore di un legame che non smette mai di esserci.*

A Francesco. *Mio cugino e figlioccio, con cui condivido non solo il nome di nostro nonno, ma anche un legame che, nonostante la differenza d'età, è sempre stato autentico e speciale. In te rivedo la stessa curiosità e la stessa luce negli occhi che rendono ogni momento insieme sincero e pieno di affetto. Crescendo, potremo percorrere strade diverse, ma mi auguro che resteremo sempre un punto di riferimento l'uno per l'altro, sperando di conservare nel tempo un legame forte e vero, capace di rispecchiare quello che ha unito i nostri padri.*

A Giovanni e Giuseppe. *Con Voi ho condiviso praticamente ogni estate della mia vita. Siamo cresciuti insieme, tra giornate interminabili, risate e ricordi che porterò sempre con me. Abbiamo fatto tutto insieme, passo dopo passo, dall'infanzia ad oggi, costruendo un legame che va oltre il tempo e le distanze. Ci siamo sempre l'uno per l'altro, e per quanto il tempo passi, so che le nostre risate continueranno ad accompagnarci, proprio come hanno sempre fatto.*

A Francesca *la mia complice di sempre. Ci siamo conosciuti tra i banchi del liceo, quando tutto doveva ancora iniziare, e da allora abbiamo condiviso ogni tappa importante del nostro percorso, affrontando insieme sogni, sfide e cambiamenti. Questi tre anni universitari sono stati un capitolo stupendo, fatto di esperienze, risate, viaggi e momenti che porterò sempre con me. In un tempo in cui tutto cambia, una delle poche certezze rimaste sei tu: la stessa presenza costante, lo stesso sguardo sincero, lo stesso equilibrio che continua a rendere ogni giorno migliore. Grazie per esserci, sempre, con la stessa autenticità di quando tutto è cominciato.*

A Fabio. *Che più che un amico è da sempre un fratello. Ci conosciamo da una vita, abbiamo condiviso ogni fase del nostro percorso dalle elementari all'università passando per mille esperienze, risate e momenti che resteranno per sempre. Se guardo indietro, in ogni ricordo importante ci sei tu. Grazie per essere, ancora oggi, una delle certezze più belle della mia vita.*

A Martina, Aurora ed Elisa. *le mie amiche, i miei bro. Grazie per aver condiviso con me questa esperienza fino in fondo, per esserci sempre, con la vostra presenza, la vostra ironia e quella leggerezza che rende ogni cosa più bella.*

A Gaetano, Antonio M., Antonio Z. e Dario. *In questi tre anni siete stati una parte fondamentale della mia vita. Abbiamo condiviso di tutto — serate interminabili, risate, partite, cene improvvisate, chiacchierate fino a notte fonda e momenti che resteranno per sempre nostri. Abbiamo fatto così tante cose insieme che è impossibile raccontarle tutte, e probabilmente per alcune non si potrebbe nemmeno. Abbiamo imparato a confidare l'uno nell'altro, ad affrontare insieme le difficoltà e a goderci ogni attimo di leggerezza. Non sono mancati per ognuno di noi dei periodi complicati, ma non saranno mai tanti quanto quelli belli che abbiamo vissuto e che vivremo ancora. Spero che, con i nostri pregi e i nostri difetti, riusciremo sempre a restare così uniti.*

A Giggi, Luigi e Giovanni. *Amici di sempre, con cui sono cresciuto e con cui tutt'oggi condivido tanto. Siete una parte importante della mia vita, e per voi ci sarò sempre, come so che voi ci sarete per me.*

A Giovanni, Simone, Francesco ed Emanuele. *Chiamarvi "colleghi" sarebbe davvero riduttivo. In questi anni siamo diventati una famiglia: abbiamo studiato insieme, viaggiato, condiviso centinaia di uscite e, soprattutto, abbiamo saputo ridere di tutto soprattutto di noi stessi. Fare i cazzoni è probabilmente ciò che ci riesce meglio, ma è proprio in quelle risate che si nasconde la forza del nostro legame. Al di là di questo titolo, la mia conquista più grande siete voi: aver trovato amici veri, con cui ogni momento diventa speciale. Mi avete insegnato che non conta da quanto tempo si conosce una persona, ma quanto sia facile sentirsi a casa con le persone giuste.*

A Davide, Federico, Rosario, Benedetta e Mario. Grazie per i momenti condivisi, per le risate, la collaborazione e per aver reso questo percorso più leggero e piacevole da vivere. Sono felice di averlo condiviso con voi.

Agli amici di sempre, di ognuno di voi potrei scrivere un pensiero, un ricordo o un momento che mi porterò dietro per sempre, ma non basterebbero le pagine per raccontarli tutti. Con voi sono cresciuto, ho riso, ho sbagliato, ho imparato. L'amicizia che mi lega a ciascuno di voi è una delle cose di cui vado più fiero nella mia vita, perché siete parte di me e di tutto ciò che mi ha reso la persona che sono oggi.